NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

PATRÍCIA ALEXANDRA GONÇALVES RODRIGUES

BSc in Electrical and Computer Engineering

NON-COOPERATIVE CONTROL OF A SHUTTLE DRONE FOR MANNED DRONE CAPTURE

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon
March 31, 2023

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

NON-COOPERATIVE CONTROL OF A SHUTTLE DRONE FOR MANNED DRONE CAPTURE

PATRÍCIA ALEXANDRA GONÇALVES RODRIGUES

BSc in Electrical and Computer Engineering

Adviser: Bruno Guerreiro
Assistant Professor, NOVA School of Science and Technology | FCT NOVA

Examination Committee:

Chair: Doctor Ana Inês da Silva Oliveira
Assistant Professor, NOVA School of Science and Technology | FCT NOVA

Members: Doctor Fernando José Almeida Vieira do Coito
Associate Professor, NOVA School of Science and Technology | FCT NOVA

Doctor Bruno João Nogueira Guerreiro
Assistant Professor, NOVA School of Science and Technology | FCT NOVA

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon
March 31, 2023

Non-cooperative control of a shuttle drone for manned drone capture

Copyright © Patrícia Alexandra Gonçalves Rodrigues, NOVA School of Science and Technology, NOVA University Lisbon.
The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

Acknowledgements

First I would like to thank FCT NOVA for providing me with the opportunities and tools to achieve my best academic potential.

I would also like to show my gratitude to my dissertation advisor, Professor Bruno Guerreiro, for the help and guidance throughout the development of this dissertation.

I would like to thank my friends Francisco, Inês, and Patrícia for making the work less heavy with their company. I'm especially thankful for my girlfriend Daniela, who had the patience to hear my failures and the excitement when I had my successes.

I would like to thank my parents for supporting me on this journey, my brother Nuno for the understanding of someone that has gone through the same experience, and for my brother Ricardo who always gives us discussion topics with his brilliant ideas.

Finally, I would like to give recognition to the one that has always been there with me throughout my degree and this thesis, my pet bunny, Bagheera, that is always ready to provide me with a distraction in the form of mischief.

This work was partially funded by the FCT projects CAPTURE (PTDC/EEI-AUT /1732/2020) and CTS (UIDB/EEA/00066/2020).

"At the end of the day, we can endure much more than we think we can." (Frida Kahlo)

Abstract

Drones have become increasingly popularized in recent years, be it in military use or as a hobby for the general public. The increase in usage of drones poses a challenge to security of zones where flying is monitored and prohibited for everyone, such as airports. Unmanned aerial vehicles show great promise in tackling problems such as this, where they must capture a drone in a set environment.

This thesis addresses this problem by studying pursuit-evasion games, human-in-the-loop as well as non-linear and predictive control in order to create an unmanned aerial vehicle capable of pursuing and capturing a manned drone.

The full models for the ground vehicle and the RUAV were done, as well as two simple versions of the fixed-wing UAV, and one simplified version of the RUAV. For the RUAV two simple internal controllers were created, in order to produce the necessary forces and torques. Next, a NMPC controller was created for each vehicle, then a simulation of both vehicles with NMPC was done for joint-tracking, and finally for the differential game. Lastly the human-in-the-loop was added to the NMPC of the fixed-wing, and then to the differential game between the fixed-wing and the RUAV.

The results show that a NMPC controller is effective at handling dynamic models and aggressive changes in trajectory while still accounting for the physical constraints imposed. The differential game used provided the pursuit-evader scenario desired, where it was possible to see various interactions between vehicles. Finally, the human-in-the-loop was integrated with the fixed-wing model.

Keywords: Human-in-the-loop, Model Predictive Control, Non-linear MPC, Differential games, Pursuit-Evasion Games, Unmanned Aerial Vehicle

Resumo

Drones têm-se tornado populares em recentes anos, quer para uso militar quer para o uso pessoal do público. Este aumento no uso de drones gera a necessidade de segurança em zonas onde veículos aéreos são monitorizados ou proibidos de voar, como por exemplo aeroportos. Veículos aéreos não tripulados apresentam uma grande promessa em combater este tipo de problemas onde é necessário capturar outro veículo num ambiente designado.

Esta dissertação pretende desenvolver uma solução para este problema começando por estudar jogos diferencias, de perseguição-evasão, humanos na malha de controlo, bem como controlo não linear e preditivo, de forma a criar um veículo aéreo não tripulado que seja capaz de perseguir e capturar um drone comandado por um ser humano.

Foram criados modelos completos do UGV e do RUAV, tal como duas versões simples do avião, e uma versão simplificada do RUAV. Para o RUAV foram desenvolvidos dois controladores simples capazes de devolver as forças e torques necessários. De seguida foi criado um controlador NMPC para cada veículo, depois foi simulado controladores NMPC para ambos os veículos em seguimento conjunto e finalmente para o jogo diferencial. Por último o ser humano foi incorporado no controlador NMPC do avião, seguido pela integração do controlador no jogo diferencial entre o avião e o RUAV.

Os resultados demonstraram que um controlador NMPC é eficaz a suportar modelos dinâmicos com mudanças agressivas de trajetória, considerando as restrições físicas impostas. O jogo diferencial ofereceu um cenário perfeito para a mentalidade desejada de perseguição-evasão, onde foi possível ver várias interações entre os veículos. Finalmente o ser humano foi incorporado no modelo e controlo do avião.

Palavras-chave: Humanos na malha de controlo, Controlo preditivo baseado em modelos, MPC não linear, Jogos diferenciais, Jogos perseguição-evasão, Veículo aéreo não tripulado

Contents

List of Figures x

List of Tables xiii

Acronyms xiv

Symbols xv

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Proposed solution . . . 2
- 1.4 Report organization . . . 2

2 Background and related work 4
- 2.1 Drone Control . . . 4
- 2.2 Human-in-the-loop . . . 9
- 2.3 Differential games . . . 11

3 Modeling and basic control 12
- 3.1 Reference frames . . . 12
- 3.2 Rotary-wing UAV . . . 13
  - 3.2.1 RUAV Model . . . 13
  - 3.2.2 Internal controller . . . 15
  - 3.2.3 Simplified RUAV model . . . 17
- 3.3 Target Models . . . 18
  - 3.3.1 Unmanned Ground vehicle . . . 19
  - 3.3.2 Fixed-wing UAV . . . 20
  - 3.3.3 Fixed-wing UAV with acceleration . . . 21
- 3.4 Simulation . . . 21
  - 3.4.1 Attitude control . . . 21

3.4.2 Non-Linear internal controller . . . 22
3.4.3 UGV . . . 23
3.4.4 Connecting the vehicles . . . 24

4 NMPC for joint trajectory tracking **27**
4.1 Non-Linear control of target vehicles . . . 27
4.1.1 Ground vehicle . . . 28
4.1.2 Fixed-wing UAV . . . 30
4.2 Non-Linear control of the rotary UAV . . . 33
4.3 Joint tracking NMPC . . . 36

5 NMPC-based Pursuit-Evasion Game **41**
5.1 Shuttle drone vs. Ground Target . . . 43
5.2 Shuttle drone vs. Fixed-wing Target . . . 46
5.3 Shuttle drone vs. Fixed-wing Target with Acceleration Control . . . 49

6 Human-in-the-loop **59**
6.1 NMPC for a fixed-wing UAV . . . 60
6.2 Pursuit-evasion Game with HIL . . . 63

7 Conclusions and Future work **67**
7.1 Future Work . . . 68

Bibliography **69**

List of Figures

2.1 Simplified model predictive control 6

3.1 Inertial and body frame referential for the ground vehicle 13
3.2 Inertial and body frame referential for the aerial vehicle. 13
3.3 Referential for the Φ_a angles of the RUAV 14
3.4 Attitude control. 15
3.5 Mellinger controller 18
3.6 Simplified RUAV control. 18
3.7 Generalized coordinates for a unicycle 19
3.8 Unmanned ground vehicle model 20
3.9 Fixed-wing UAV model. 20
3.10 Fixed-wing UAV model with acceleration. 21
3.11 Plot of desired and obtained ψ and θ 22
3.12 Position evolution for the RUAV Mellinger controller 23
3.13 Plot for the 3D coordinates of the UAV with a Mellinger controller 24
3.14 UGV state evolution 24
3.15 Movement of the UGV with y in function of x. Blue arrows represent the vehicles orientation 25
3.16 Plot for the UGV and RUAV state evolution 25
3.17 2D plot for the UGV and RUAV 25
3.18 3D plot for the UGV and RUAV 26

4.1 NMPC control 27
4.2 State evolution for the ground vehicle NMPC. 29
4.3 x-y plane for the ground vehicle NMPC 29
4.4 Input evolution for the NMPC (Ground vehicle) 30
4.5 Time of each iteration for the UGV NMPC algorithm 31
4.6 Position states for the fixed-wing UAV 32
4.7 Linear velocity states for the fixed-wing UAV 32
4.8 Angle states for the fixed-wing UAV 33

4.9 Input evolution of the acceleration for the fixed-wing UAV 33
4.10 Input evolution of the angular velocity for the fixed-wing UAV 34
4.11 3D plot of the position for the fixed-wing UAV 34
4.12 Time of each iteration for the NMPC fixed-wing UAV with acceleration . . 35
4.13 State evolution for the RUAV NMPC 36
4.14 Input evolution for the RUAV NMPC 36
4.15 xy-axis plot for the RUAV NMPC 37
4.16 Time of each iteration for the RUAV NMPC 37
4.17 State evolution of the position states (connected NMPC RUAV and UGV). . 38
4.18 Control evolution for the RUAV connected to the UGV. 38
4.19 x-y plane of the vehicles with RUAV following UGV. 39
4.20 Plot for time in function of iteration for the connected UGV and RUAV . . 39

5.1 Schematic for the differential game between heterogeneous vehicles 43
5.2 State evolution (UGV vs. RUAV) - Equal constraint 44
5.3 State evolution (UGV vs. RUAV) - Faster UGV 44
5.4 State evolution (UGV vs. RUAV) - Faster RUAV 45
5.5 Input evolution of UGV - Equal constraint 45
5.6 Input evolution of UGV - Faster UGV 45
5.7 Input evolution of UGV - Faster RUAV 46
5.8 Input evolution of RUAV - Equal constraint 46
5.9 Input evolution of RUAV - Faster UGV 46
5.10 Input evolution of RUAV - Faster RUAV 47
5.11 XYZ plot (UGV vs. RUAV) - Equal constraints 47
5.12 XYZ plot (UGV vs. RUAV) - Faster UGV 47
5.13 XYZ plot (UGV vs. RUAV) - Faster RUAV 48
5.14 Time of each iteration (UGV vs. RUAV) - Equal constraints 48
5.15 Time of each iteration (UGV vs. RUAV) - Faster UGV 48
5.16 Time of each iteration (UGV vs. RUAV) - Faster RUAV 49
5.17 Distance between vehicles (UGV vs. RUAV) - Equal constraints 49
5.18 Distance between vehicles (UGV vs. RUAV) - Faster UGV 49
5.19 Distance between vehicles (UGV vs. RUAV) - Faster RUAV 50
5.20 State evolution (FW UAV vs. RUAV) - Equal constraints 51
5.21 State evolution (FW UAV vs. RUAV) - Faster FW UAV 51
5.22 State evolution (FW UAV vs. RUAV) - Faster RUAV 51
5.23 Input evolution (FW UAV) - Equal constraints 52
5.24 Input evolution (FW UAV) - Faster FW UAV 52
5.25 Input evolution (FW UAV) - Faster RUAV 52
5.26 Input evolution (RUAV) - Equal constraints 53
5.27 Input evolution (RUAV) - Faster FW UAV 53
5.28 Input evolution (RUAV) - Faster RUAV 53

5.29 XYZ plot (FW UAV vs. RUAV) - Equal constraints 54
5.30 XYZ plot (FW UAV vs. RUAV) - Faster FW UAV 54
5.31 XYZ plot (FW UAV vs. RUAV) - Faster RUAV 54
5.32 Time of each iteration (FW UAV vs. RUAV) - Equal constraints 55
5.33 Time of each iteration (FW UAV vs. RUAV) - Faster FW UAV 55
5.34 Time of each iteration (FW UAV vs. RUAV) - Faster RUAV 55
5.35 Distance between vehicles (FW UAV vs. RUAV) - Equal constraints 55
5.36 Distance between vehicles (FW UAV vs. RUAV) - Faster FW UAV 55
5.37 Distance between vehicles (FW UAV vs. RUAV) - Faster RUAV 56
5.38 Position and yaw (FW w/ acceleration vs. RUAV) - Faster Fixed-Wing . . . 56
5.39 Position and yaw (FW w/ acceleration vs. RUAV) - Faster RUAV 56
5.40 Input Evolution of the FW (FW w/ acceleration vs. RUAV) - Faster Fixed-Wing 57
5.41 Input Evolution of the FW (FW UAV w/ acceleration vs. RUAV) - Faster RUAV 57
5.42 Input Evolution of the RUAV (FW UAV w/ acc. vs. RUAV) - Faster FW . . . 57
5.43 Input Evolution of the RUAV (FW UAV w/ acc. vs. RUAV) - Faster RUAV . 57
5.44 XYZ plot (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing 57
5.45 XYZ plot (FW UAV w/ acceleration vs. RUAV) - Faster RUAV 57
5.46 Time of each iteration (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing 58
5.47 Time of each iteration (FW UAV w/ acceleration vs. RUAV) - Faster RUAV . 58
5.48 Distance between vehicles (FW UAV w/ acc. vs. RUAV) - Faster FW 58
5.49 Distance between vehicles (FW UAV w/ acc. vs. RUAV) - Faster RUAV . . . 58

6.1 State evolution of the position for the NMPC with HIL 60
6.2 State evolution of the velocity for the NMPC with HIL 61
6.3 State evolution of the angles for the NMPC with HIL 61
6.4 Input evolution of the acceleration for the NMPC with HIL 62
6.5 Input evolution of the angular velocities for the NMPC with HIL 62
6.6 3D plot for the NMPC with HIL 62
6.7 Time of each iteration for the NMPC with HIL 63
6.8 XYZ plane for the differential game with HIL (Unchanged values) 63
6.9 State evolution for the differential game with HIL 64
6.10 Input evolution of the fixed-wing vehicle for the differential game with HIL 64
6.11 Input evolution of the RUAV for the differential game with HIL 65
6.12 XYZ plot for the differential game with HIL 65
6.13 Distance between vehicles for the differential game with HIL 65
6.14 Time of each iteration for the differential game with HIL 66

List of Tables

3.1 Constant values for the attitude control 21
3.2 Values of the gains used in the attitude control 22
3.3 Table with the position controller gains. 23

4.1 NMPC - UGV gains 28
4.2 NMPC - fixed-wing UAV gains for $T_s = 0.1$s and $N = 15$ 31
4.3 NMPC - RUAV and UGV gains for $T_s = 0.1$s and $N = 15$ 35
4.4 Simulation times for each NMPC case 40

5.1 Cost matrices values for the UGV and RUAV differential game 43
5.2 Input constraints for the differential game (UGV vs. RUAV) 44
5.3 Cost matrices values for the fixed-wing and RUAV differential game 47
5.4 Input constraints (fw UAV vs. RUAV) 50
5.5 Cost matrices for the fixed-wing w/ acceleration and RUAV differential game. 50
5.6 Input constraints (fw UAV w/ acceleration vs. RUAV). 52
5.7 Total and individual runtimes for the differential game 56

Acronyms

CPS Cyber-Physical Systems 9

ECEF Earth-Centered Earth-Fixed 12
ECI Earth-Centered Inertial 12
ENU East-North-Up 12

FMD Fixed-wing Manned Drone 2

HIL Human-in-the-Loop 9
HMM Hidden Markov Model 9

MPC Model Predictive Control 4

NED North-East-Down 12
NMPC Non-linear Model Predictive Control 5

OCP Optimal Control Problem 60

PEG Pursuit-Evasion Game 11
POMPD Partially Observable Markov Decision Process 10

RUAV Rotary-wing UAV 2

UAV Unmanned Aerial Vehicle 1
UGV Unmanned Ground Vehicle 2, 11

Symbols

a_r	Rotational air friction coefficient 14
a_t	Linear air friction coefficient 14
B_a	Body frame of the UAV 12, 13
B_g	Body frame of the UGV 12
$\{x_{ba}, y_{ba}, z_{ba}\}$	Base vectors of the body frame for the UAV 13
$C = \begin{bmatrix} x_C & y_C & z_C \end{bmatrix}^T$	Intermediate coordinate frame 13
E	Jacobian matrix of R_{IB} 14
e_{pos}	Position error for the UAVs internal controller 16
e_{rot}	Rotation error for the UAVs internal controller 17
e_{vel}	Velocity error for the UAVs internal controller 16
e_ω	Angular velocity error for the UAVs internal controller 17
f_a	Aerodynamic force 14
F_{des}	Desired force for the UAVs internal controller 16
f_e	External force 14
γ	Flight-path angle angle 20
I	Inertia matrix 14
$\{I_{xx}, I_{yy}, I_{zz}\}$	Principal moments of inertia, diagonal elements of the inertia matrix 14
V_x	UGV input - desired linear velocity 19
ω_z	UGV input - desired angular velocity 19

K_{pos}, K_{vel}	Position and velocity gain matrices for the UAVs internal controller 16
K_{rot}, K_{ω}	Rotation and angular velocity gain matrices for the UAVs internal controller 17
$P_a = \begin{bmatrix} x_a & y_a & z_a \end{bmatrix}^T$	Linear position vector of the UAV 13
θ	Pitch rotation angle 13
$\Phi_a = \begin{bmatrix} \phi_a & \theta_a & \psi_a \end{bmatrix}^T$	ZYX Euler angles vector of the UAV 13
ϕ	Roll rotation angle 13
τ_a	Aerodynamic torque 14
τ_e	External torque 14
T_{rot}	Propellers thrust 15
$V_a = \begin{bmatrix} u_a & v_a & w_a \end{bmatrix}^T$	Linear velocity vector of the UAV 13
$\omega_{B_a} = \begin{bmatrix} p_a & q_a & r_a \end{bmatrix}^T$	Angular velocity vector of the UAV 13
ψ	Yaw rotation angle 13

1

Introduction

1.1 Motivation

The interest in drones has been growing from a research and military standpoint. Drones are being increasingly used for aerial surveillance, 3D mapping, among other things. Doing such tasks autonomously requires reliable navigation from the drones, particularly in unknown areas. Moreover, Unmanned Aerial Vehicles (UAVs) are becoming more developed for scenarios where it is advantageous to deploy an unmanned vehicle that has a high level of autonomy to carry out tasks that can be done without human supervision.

However, these devices aren't used only in the research and industry fields, as many people have incorporated drones in their day to day lives, usually as a hobby. Although there is nothing wrong with using drones for fun there are some issues, such as safety and privacy, that need to be respected. A good example of that are airports, where the sighting of a drone within the airport's area can cause a temporary shutdown, leading to many delays in the flights, like in January of 2023 where a drone sighting in a Dublin Airport caused a temporary suspension of the flight operations [1]. Another problem arises when drones are flown in neighbourhoods. If not regulated drones can be used to go over houses and backyards, being able to look into people's homes, permitting the drone's owner to see or record what is happening inside someone's property, which is a clear breach of privacy. This is why many countries, including Portugal, have laws and regulations on this subject. Some of the rules applied to the piloting of drones in Portugal set forth by ANAC (Autoridade Nacional da Aviação Civil) include not flying over crowds of people, not flying above 120 meters, and not dropping materials from the drone. Moreover, it is necessary to go to the authorities in order to get assigned a category to be able to fly the drone legally, where each category has a slightly different sets of rules, depending on what the drone is being used for [2].

The question comes when someone flies a drone where they should not, be it by accident or on purpose, and how to best capture it. The focus of this dissertation is answering that question, by using an unmanned aerial vehicle with a system able to model the human being and predict their next move so that it can apprehend its target.

1.2 Problem Statement

One of the challenges of this thesis is how to create a pursuit-evasion game between two drones, where one is a Rotary-wing UAV (RUAV) and the other is a Fixed-wing Manned Drone (FMD).

For the RUAV there is a need to consider what type of control to implement in order for it to have smooth and fast responses to keep up with the FMD. When considering a FMD the main challenge is that humans are difficult to model and turn into mathematical equations, therefore a lot of thought must go into how and where to include the human in the loop. Although the FMD may have disadvantages, such as slower responses to changes in environment or more difficulty in surveying its surroundings, it is still a system that relies on human intelligence and ingenuity to evade.

Finally, it is important to consider how best to implement differential games in order to create an environment where the RUAV is pursuing the FMD that is in turn evading.

1.3 Proposed solution

To be able to create an RUAV that can pursue another vehicle, control strategies that can emulate and affect the RUAV smoothly and accurately are very important. The non-linear model predictive control will be studied as this is a type of control that can handle non-linear and unpredictable systems.

To incorporate a human-in-the-loop, a delay will be introduced to the FMD to take into consideration the reaction time of a pilot and the time it takes between perceiving the information, processing it, and acting accordingly.

Since the RUAV has the objective of capturing a non-cooperative drone, a differential game, as pursuit-evasion game, will be developed. For a first step the game will include only two players: the RUAV and the FMD.

1.4 Report organization

Chapter 2 explains some of the key subjects used in the development of this thesis, as well as shine a light on previous related works, focusing on model predictive control, human-in-the-loop, and differential games. In Chapter 3 the Unmanned Ground Vehicle (UGV) and the aerial vehicles are modelled and in the case of the RUAV some internal controllers will be developed. Chapter 4 shows the creation of a NMPC for joint trajectory tracking where at first both vehicles follow a reference and later the RUAV follows the UGV, who is still following the reference. On Chapter 5 we can see the development of the differential game between the shuttle drone and the target vehicles, while on Chapter 6 the human is added to the loop, first for the fixed wing with acceleration with a NMPC controller and then to the differential game between same vehicle as before and with the

RUAV. Finally, Chapter 7 concludes on all the work presented as well as appoint some interesting future developments.

2

Background and related work

The interest in UAVs has been increasing and as a result there has been a rise in the funding of projects that use UAVs in innovative and advantageous ways. Drones have come to be employed in many areas, like in commerce, the military, and agriculture. One example of the usage of drones is in the event of a natural disasters, and in some cases the prevention of one. The authors in [3] show various examples of how drones can be used in this scenario, like mapping areas hit by earthquakes to access possible survivors, the state of the buildings, and where to dispatch resources. Or in the case of a flood where the drone can have a preventive side by tracking bodies of water to see their water level, and a more active side by helping predict the areas at most risk, where to evacuate civilians, among other things. The last example is a well known one, forest fires. Where the drone can help detect hot-spots and provide firefighter forces with real time information to help manage and optimize the combat against fires.

Many industries use drones in their day-to-day business, for example, the mining industry. UAVs have been used in surface mining to do ore control, 3D mapping of the mine's environment, blasting management, and tailing stability, etc. Although the usage of drones in underground mines is still limited due to the lack of access to GPS, and wireless signals, the study of these drones in this case is promising [4].

Drone control is very important to achieve these results, as well as for this dissertation. This chapter will start by studying the types of control that can be used, namely trajectory tracking and Model Predictive Control (MPC). Section 2.2 focuses on reviewing the role of a human and how to predict their actions. Finally, there will be an overview of differential games in Section 2.3.

2.1 Drone Control

The control of UAVs has been evolving as more research in this area has been happening. An example of a type of control often used in UAVs is the proportional–integral–derivative (PID) controller. In [5] the authors present three different architectures of a cascade control system using different combinations of PIDs in order to control the drone's attitude,

finding that the PID did a good job of controlling the system. Another way was studied in [6] by using the sliding mode based on backstepping approach, which ensures Lyapunov stability, robustness, and handling of the system nonlinearities while considering the nonholonomic constraints.

One in particular has piqued our interest and has shown to satisfy the need for control of non-linear systems that consider constraints: Non-linear Model Predictive Control (NMPC) [7].

Model predictive control allows for the representation of complex and dynamic systems with multiple inputs and outputs while taking into account sets of constraints created by either the system itself or its surroundings [8]. To do so an optimization problem is developed that predicts over a finite horizon (N) the optimal evolution of the state while considering optimality criterion, which results in the optimal control sequence U^*. Although the whole sequence is calculated, only the first element is applied to the system. To compensate for possible modelling errors and disturbances acting on the system a new state measurement is taken at the next sampling instance, and the whole process is repeated, introducing feedback to the procedure. Hence the fixed prediction horizon is shifted or receded over time, which gives it its name, receding horizon control. The process for this on-line optimal control can be seen in Algorithm 1 [9][10].

In [11] the use of MPC control for the problem of trajectory tracking is studied. First there was a non-linear MPC approach that led to a non-convex optimization problem, followed by a linear MPC which reduced the computational effort while maintaining a good performance. It is necessary to take into account that linear MPC is only viable for points close to the reference trajectory.

The authors of [12], [13], and [14] review the MPC for the control of heating ventilation and air-conditioning (HVAC) systems. They conclude that, overall, MPC outperforms other conventional controllers in saving energy, minimizing cost consumption, maximizing thermal comfort, among others. One downside was that the MPC is very specific to the building it is predicting for, since the algorithm has to be customized to fit the specific building features.

Although it is the more widely used of the two controls, it relies on making linear approximations of the nonlinear dynamics and constraints, while the NMPC has a more direct and explicit approach as seen in [15].

Non-linear MPC is becoming more frequently used because of its ability to work with non-linear and unsteady systems that have processes with wide swings in operation that make it difficult to linearize [16]. It has been used on systems with somewhat slow dynamics, such as chemical processes. However, with the advancements in technology, computers are becoming faster and non-linear MPC is broadening to large scale multi-agent control problems [17].

In most control problems the objective is to create stabilizing feedback that minimizes the performance index while still fulfilling the constraints on the inputs and states. It would be preferable to obtain a closed solution for the feedback law that got these results.

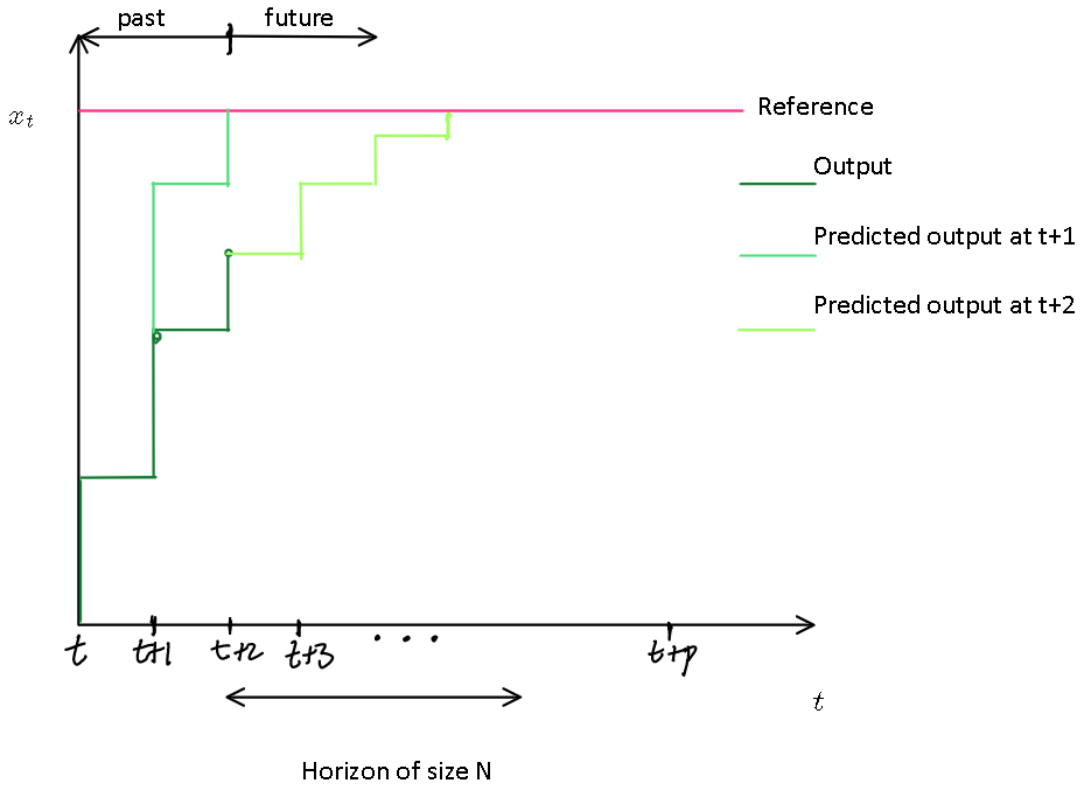


Figure 2.1: Simplified model predictive control

Unfortunately, most of the times the optimal feedback law cannot be found analytically, even when it is unconstrained. This is where model predictive control comes in, where the algorithm repeatedly solves an open-loop optimal control problem influenced by the systems dynamics as well as constraints. The control law is implemented by only using the first optimal input obtained from the control sequence and feeding it to the system [18]. Figure 2.1 shows a very simple version of this type of control.

Taking into account the output obtained at time t, the controller predicts the future actions of the system over the prediction horizon and determines which input to apply to the next instance. Since there are always disturbances and discrepancies with the model, the real system behaviour will differ from the predicted one. In order to implement a feedback mechanism, the input that is obtained will only be applied in the next sampling instance. Therefore, the prediction and calculation of the input is repeated, with the horizon moving forward.

Non-linear control systems are often described by differential equations,

$$\dot{x}(t) = f(x(t), u(t)), \qquad x(0) = x_0$$

where

$$u(t) \in U, \forall t \geq 0 \qquad x(t) \in X, \forall t \geq 0$$

in which $x(t)$ and $u(t)$ are the state and input vectors, respectively, U and X are the feasible input and state values [19]. The cost function for the NMPC is given by

$$J_t := J_{t\to N}(x_0, u) = p(x_N) + \sum_{k=t}^{N-1} q(x_k, u_k) \tag{2.1}$$

where the time interval is from the current moment (t) to the end of the horizon (N). The final cost function ($p(x_N)$) and intermediate cost function ($q(x_N)$) are given by

$$p(x_N) = x_N^T P x_N \quad \text{and} \quad q(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k \tag{2.2}$$

where P, Q, and R are matrices that represent the weight of the given variable. For the input the smaller the matrix's elements are, the more input we are able to expend. For the states matrix, the bigger they are the more the system will follow the reference given. The state related matrices P and Q have the size nxm, where n is the number of states we want to control, ranging from 1 to all the states (n_x), and m is given by n_x. So, if we wanted to control all the states then the matrices would be n_x by n_x. Similarly, the input related matrix R is given by pxn_u, where p can be 1 to n_u. Besides these following three things should be considered:

1. $P = P^T \geq 0$
2. $Q = Q^T \geq 0$
3. $R = R^T > 0$

The optimal control problem can be defined as:

Optimal control problem: Find the optimal state control sequence u_k^* for $k \in 0, ..., N-1$ that drives the system along a state trajectory x_k^* for $k \in 0, ..., N$ according to the system dynamics and within input and state constraints, such that the specified performance index J_0 is minimized relative to a reference output signal $\bar{y}_k$. Mathematically this is given by:

$$\begin{aligned} \min_{u_0,\ldots,u_{N-1}} \quad & J_0 = (y_N - \bar{y}_N)^T P (y_N - \bar{y}_N) + \sum_{k=0}^{N-1} (y_k - \bar{y}_k)^T Q (y_k - \bar{y}_k) + u_k^T R u_k \\ s.t \quad & x_{k+1} = f(x_k, u_k), \forall k = 0, \ldots, N-1 \\ & y_k = C x_k \\ & x_0 = z_0 \end{aligned} \tag{2.3}$$

where z_0 is the initial condition given by the current state, C is the output matrix, and $f(x_k, u_k)$ is the equality constraint. If we stack the solution in Equation (2.3) from moment t to N, we can represent the cost function in the batch approach which yields:

$$\begin{aligned} \min_{U} \quad & J_0 = (Y - \bar{Y})^T \bar{Q} (Y - \bar{Y}) + U^T \bar{R} U \\ s.t \quad & F(X, U) = 0 \end{aligned}$$

Algorithm 1 Receding horizon control

Require: Current state x

1: Consider the initial condition x_0 is the current state x_{t_k}.
2: Obtain the optimal control sequence U^* solving the non-linear optimal control problem by minimizing the cost function over the prediction horizon.
3: If the problem is unfeasible then terminate the algorithm.
4: Apply the first control action from U^* to the system, $u_{t_k} = u_0^*$
5: Repeat from step 1.

where $\bar{Y}$ is the reference we want followed, Y is given by $Y = HX$ with $X = \begin{bmatrix} x_0 & \cdots & x_{N-1} & x_N \end{bmatrix}^T$, H is a block-diagonal matrix where the non-zero elements have the value of the output matrix C, the input is given by $U = \begin{bmatrix} u_0 & \cdots & u_{N-2} & u_{N-1} \end{bmatrix}^T$, the optimal input is $U^* = \begin{bmatrix} u_0^* & \cdots & u_{N-2}^* & u_{N-1}^* \end{bmatrix}^T$, $\bar{Q} = diag(Q, Q, ..., P)$, $\bar{R} = diag(R, R, ..., R)$ and lastly

$$F(X,U) = \begin{bmatrix} x_0 - z_0 \\ f(x_0, u_0) - x_1 \\ ... \\ f(x_{N-1}, u_{N-1}) - x_N \end{bmatrix}. \tag{2.4}$$

To accomplish this, we use the receding horizon control algorithm which has the following steps: The initial condition instantiates the system model, which is used to predict the behaviour, with the actual systems state at that sampling instant. To solve the algorithm the Matlab function `fmincon` was used, the receding horizon is within the interval of 3 and 15, and the sampling time will be overall 0.1 seconds. For prediction it is necessary to have all state information, be it by obtaining the values through measurement or estimation. In this type of control there isn't a specific controllability assumption. Rather, the stability is checked through the initial feasibility of the optimal control problem, in other words, the input trajectory exists, and all the constraints are satisfied [18].

The ideal conditions would be having an infinite horizon in order to minimize the performance index decided by the cost function. However, to solve the open-loop optimal control it must be done on-line. Because of this it is often formulated using a finite horizon, in order to allow a real-time numerical solution of the control problem. Although we would want an infinite horizon, the larger it is the more computational weight it has. Because of this there needs to be a balance between the size of the horizon, which has to be big enough to be able to predict the trajectory with good results, and the time and effort it takes to compute the algorithm [19].

One of the big advantages for this type of control is that it is versatile. In [20] non-linear control is used in an economic setting pairing it with reinforcement learning. In [21] it is shown the development of a NMPC for the wastewater treatment process.

2.2 Human-in-the-loop

A very important aspect of this project is the human being. Humans are complex and, more often than not, unpredictable so although including humans in the control loop can be advantageous, it is not a simple task. The authors in [22] classify the combination of Human-in-the-Loop (HIL) controls into three categories:

1. Ones where the human is directly in control of the system;
2. Ones where the system monitors the humans passively and takes the appropriate actions;
3. A hybrid of the two.

While in [23] the authors believe that there are three main challenges in Cyber-Physical Systems (CPS) for HIL:

1. Being able to understand the complete spectrum types of HIL controls: To do so it is important to create a taxonomy of HIL applications based on their control. This taxonomy is the same as shown previously as the three categories of HIL.
2. The need to model human behaviour: It is necessary to employ system identification in order to create system models. Understanding which equations to use, what are the input and outputs of the system, and how to incorporate human traits in the system is incredibly complex be it physiological, psychological and environmental factors.
3. Understanding where in the loop to incorporate the human being: It is a challenge to know where in the control loop itself to integrate human input. It could be outside the loop, inside the controller, inside the system model, among other options.

In [24] the human provides input for the positions of the quadruped rescue robots front legs while the controller satisfies state constraints in order to ensure the robots stability. To do so a model predictive optimal controller that places the leg of the robot as close to where the human intended without compromising its static stability for the next leg placement is implemented. The authors of [25] study human driving behaviours and how to recognize them, hence a cognitive vehicle would need to sense data about its environment, the vehicle, the driver and the driving situation in order to create an appropriate response to the scenario. A Hidden Markov Model (HMM) is proposed to symbolize, discern and create driving patterns. According to [26] the modelling framework should have the following properties:

- Since it is impossible to measure the human's exact intention or psychological state, the model ought to be probabilistic;
- It should be a sequential process;

- It would need to handle observational error.

In the paper a Partially Observable Markov Decision Process (POMPD) is proposed. The case is made for POMPD over HMM since for the later the decision is made from a single estimation while the POMPD supplies an integrated model that incorporates hidden states, observations, and control actions. Furthermore [27] explains that POMPD allows for the representations of situations where the entire state of the system is not observable and that these hidden states are also used when planning. In [28] the subject of HIL is studied when applied to CPSs and the internet of things (IoT). They conclude that the continued development of these systems will lead to systems that adapt to the human will, instead of having intelligent IoT and CPS systems that autonomously control our environment. The authors in [29] focus on cyber-physical production systems and outlines how important it is to properly integrate humans with the technology using HIL. In [30] the focal point is on synthesising a semi-autonomous controller that is occasionally corrected by a human operator. In a different approach [31] studies the effect of trust in automation, namely in a shared control of a vehicle. They observe that over-trust in automation can lead to unsafe behaviours and less awareness and attention to the surroundings.

In [32] three stages for control behaviour are stipulated, the compensatory, pursuit, and precognitive control. For our case the first control is more interesting, where the human controller acts only on the error between the system output and the reference. For this type of control two models are shown, one is called the extended crossover model that approximates linear controller dynamics which is described by an equalization parameter and an equivalent time delay and is only valid near the crossover frequency. The last one is a second order model that describes human dynamics in a wider frequency range while also considering neuromuscular dynamics, called the simplified precision model. The full compensatory is the combination of these two which navigates the trade-off between stability and performance.

When dealing with dynamical systems models, a simple way of incorporating the human-in-the-loop is to introduce a delay to the vehicle that is being controlled by the pilot. This delay is meant to stand in as the reaction time (RT) of a person representing how long it takes for a human to process the information and act upon it. Reaction time can be defined as the interval of time between the appearance of the stimulus and the voluntary response given by the subject. A study involving 120 healthy medical students between the ages of 18 and 20 found that the mean for visual reaction time for sedentary medical students was 250.12±18.50 ms and for auditory reaction time it was 229.80±16.73 ms, while for medical students who regularly exercise the mean for visual and auditory reaction time, respectively, was 235.00±13.02 ms and 219.05±11.97 ms. Reaction time can be affected by gender, age, fatigue, exercise, personality types, among others, so the actual RT may vary significantly [33].

2.3 Differential games

Finally, we will discuss differential games and their importance for this work. In this thesis we will have to take into account not only one system but two, meaning our UAV and the drone controlled by a human pilot. Differential games allow us to generalize the optimal problem to two inputs and two criteria. Each player will have an objective and apply control towards that goal. Considering players A and B, the dynamical system can be given by

$$\dot{x} = f(t, x, A, B)$$

with an initial condition $x(t_0) = x_0$ and a payoff P(A,B) that can be described as

$$P(A, B) = g(\tilde{t}, x(\tilde{t})) + \int_{t_0}^{t} h(t, x, A, B)dt$$

where A has the goal to maximize the P(A,B) while B wants to minimize it. In differential games both players are allowed to choose any control function as long as its values are within given sets. The difficulty is that in instance t both players are aware of their previous control actions as well as their opponents, but don't know what the future control of the adversary is going to be. Assuming that $\tilde{A}$ and $\tilde{B}$ are existing controls so that

$$P(A, \tilde{B}) \leq V \leq P(\tilde{A}, B)$$

where $V = P(\tilde{A}, \tilde{B})$. Taking this into account some statements can be made. If player B chooses $\tilde{B}$, independently of the actions of A, then the payoff will be less than V. Similarly, if A chooses $\tilde{A}$ the payoff will be greater than V. Therefore, V is the payoff of when both players are doing their best and can be denominated as the value of the game. The $(\tilde{A}, \tilde{B})$ pair is the saddle point which represents the optimal strategy for both players [34]. In particular we will discuss Pursuit-Evasion Game (PEG)s, where pursuers have the goal of locating and catching the other players and the evaders try to elude them. In [35] the type of PEG is a zero-sum game, where all the players payoffs sum up to zero. In this article a non-cooperative differential game between a target vehicle and a shuttle vehicle is developed using a NMPC. Both [36] and [37] focus on cooperative differential games where the players maximize the sum of their payoffs, and the game develops along a cooperative trajectory. On the other hand [38] shows the use of differential games in an economic setting. The authors of [39] combined PEGs with map building in a probabilistic approach while using a team of UAV and Unmanned Ground Vehicle (UGV) in a hierarchical hybrid system architecture to pursue a team of evaders, inside an enclosed unknown space. In [40] a distributed formation is studied using differential game theory. In this paper UAVs are able to communicate with one another by using a directed graph. Their individual objective is to minimize their terminal formation errors, the divergence of terminal velocity with the other UAVs, and their control efforts while still maintaining formation.

3

Modeling and basic control

This chapter starts by introducing the modeling concepts for the RUAV drone as well as the target vehicles used in this thesis, defining the coordinate frames used to deduce these models. It also shows the development of two internal controllers for the RUAV, as well as simulations of the vehicles.

3.1 Reference frames

Before modeling any of the vehicles it is important to explain and categorize the reference frames applied. Reference frames are tools employed to describe relative motions of bodies using a set of coordinates to describe positions and velocities, among others. In this case the mass center of the earth is considered as the origin of the frame, with the z-axis pointing towards the north pole and the x-y plane coinciding with the earth's equatorial plane, where the x-axis points towards the vernal equinox [1]. This frame is generally designated as the Earth-Centered Inertial (ECI) frame of reference. The ECI frame rotates along with the earth, unlike the inertial frame. Because of this it is convenient to use the Earth-Centered Earth-Fixed (ECEF) frame instead. These coordinates can be defined similarly to the ECI coordinates, with the exception of the x-axis which is now aligned with the international reference meridian. More importantly, the earth-fixed inertial frame can be defined by using a local tangent plane to the earth's surface. For aerospace vehicles the usual convention used is the North-East-Down (NED), however given the fact that we have several different vehicles, the East-North-Up (ENU) was used as the inertial frame (I), represented in Figures 3.1 and 3.2 with the axes $x_i = \begin{bmatrix}1 & 0 & 0\end{bmatrix}^T$, $y_i = \begin{bmatrix}0 & 1 & 0\end{bmatrix}^T$, and $z_i = \begin{bmatrix}0 & 0 & 1\end{bmatrix}^T$. In this convention the x-axis is pointing east, the y-axis is pointing north, and the z-axis is pointing up. The body frame is attached to the vehicle, originating at the center of mass of the object. The axes x_{B_j} and y_{B_j} are in the plane defined by the center of the vehicle, while z_{B_j} is perpendicular to that plane pointing upwards, where $j = a, g$ are for the UAV (B_a) and the UGV (B_g) respectively. For the UAV one additional

[1] The vernal equinox pertains to the intersection of the earth's orbit and equatorial plane

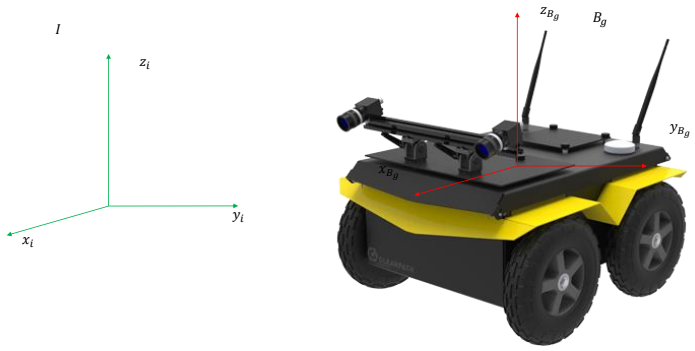


Figure 3.1: Inertial and body frame referential for the ground vehicle, adapted from [41].

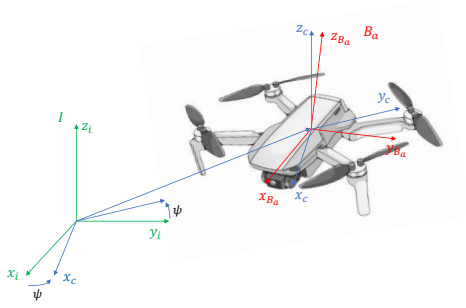


Figure 3.2: Inertial and body frame referential for the aerial vehicle.

reference frame was introduced. An intermediate coordinate frame used for computation, $C = \begin{bmatrix} x_C & y_C & z_C \end{bmatrix}^T$, which is defined as the inertial frame after a yaw rotation.

3.2 Rotary-wing UAV

3.2.1 RUAV Model

Now the RUAV will be analysed. Admitting the same inertial frame as before and $B_a = \{x_{ba}, y_{ba}, z_{ba}\}$ as the body frame of the aerial vehicle, as seen in Figure 3.2. Let $P_a = \begin{bmatrix} x_a & y_a & z_a \end{bmatrix}^T$ and $V_a = \begin{bmatrix} u_a & v_a & w_a \end{bmatrix}^T$ be the linear position and velocity in I and $\Phi_a = \begin{bmatrix} \phi_a & \theta_a & \psi_a \end{bmatrix}^T$ and $\omega_{B_a} = \begin{bmatrix} p_a & q_a & r_a \end{bmatrix}^T$, or alternatively $\omega_{B_a} = \begin{bmatrix} \omega_{a_x} & \omega_{a_y} & \omega_{a_z} \end{bmatrix}^T$, the attitude and angular velocity in B_a. In Figure 3.3 the referential for the roll (ϕ), pitch (θ), and yaw (ψ) can be seen. The first relates to the drone rotating in the front to back axis (y-z axis), the second is when it rotates in the side-to-side axis (x-z axis), and lastly when it rotates the vertical axis (x-y axis). Using the kinematic and Newton-Euler equations

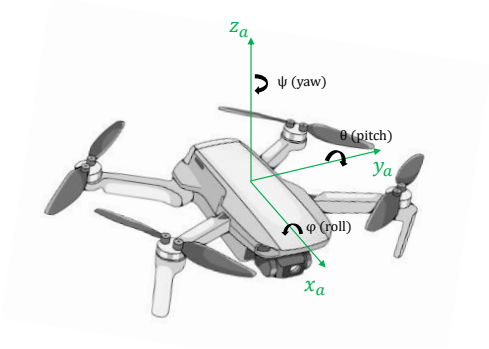


Figure 3.3: Referential for the Φ_a angles of the RUAV

the UAV model can be written as:

$$\begin{aligned}
\dot{P}_a &= V_a \\
\dot{\Phi}_a &= E(\Phi_a)^{-1}\omega_{B_a} \\
\dot{V}_a &= m^{-1}R_{IB}(\Phi_a)(f_e + f_a) - gR_{IB}(\Phi_a)^T z_i \\
\dot{\omega}_{B_a} &= I^{-1}(-\omega_{B_a} \times (I\omega_{B_a}) + \tau_e + \tau_a)
\end{aligned} \tag{3.1}$$

where m is the mass of the drone, g is the gravity's speed, I is the inertia matrix with the diagonal elements $\{I_{xx}, I_{yy}, I_{zz}\}$ that are the principal moments of inertia, and $R_{IB}(\Phi)$ and $E(\Phi)$ are the known rotation matrices and it's Jacobian respectively, using Euler ZYX angles, $R_{IB}(\Phi)$ is shown in Equation (3.2) and $E(\Phi)^{-1}$ in Equation (3.3), where for simplicity $c = \cos(\cdot), s = \sin(\cdot), t = \tan(\cdot)$.

$$R_{IB}(\Phi) = \begin{bmatrix} c\theta c\psi & c\theta s\psi & -s\theta \\ s\phi s\theta c\psi - c\phi s\psi & s\phi s\theta s\psi + c\phi c\psi & s\phi c\theta \\ c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi - s\phi c\psi & c\phi c\theta \end{bmatrix}^T \tag{3.2}$$

$$E(\Phi)^{-1} = \begin{bmatrix} 1 & s\phi t\theta & c\phi t\theta \\ 0 & c\phi & -s\phi \\ 0 & \frac{s\phi}{c\theta} & \frac{c\phi}{c\theta} \end{bmatrix} \tag{3.3}$$

The aerodynamic force (f_a) and torque (τ_a) produced by aerodynamic drag is, respectively, $f_a = \begin{bmatrix} f_{ax} & f_{ay} & f_{az} \end{bmatrix}^T = -diag(a_T, a_T, a_T)R_{IB}(\Psi)^T V_{air}$ with $V_{air} = V_a - V_w$ where V_w is the wind's velocity vector in the inertial frame, and $\begin{bmatrix} \tau_{ax} & \tau_{ay} & \tau_{az} \end{bmatrix}^T = -diag(a_R, a_R, a_R)\omega_a$, where a_t and a_r are the linear and rotational air friction, respectively. For the external force (f_e) and torque (τ_e) we have that $f_e = \begin{bmatrix} 0 & 0 & f_{ez} \end{bmatrix}^T$ and $\tau_e = \begin{bmatrix} \tau_{ex} & \tau_{ey} & \tau_{ez} \end{bmatrix}^T$. The values for these variables can be seen in Table 3.1. NMPC supports continuous models so we could proceed from here without adding any more steps to the model. In spite of this the discretization of our system is needed considering the tools we have used to simulate the vehicles behaviours. If we shorten the continuous equation of the model, we get

$$\dot{x}_a(t) = f_{a_c}(x_a(t), u_a(t)) \tag{3.4}$$

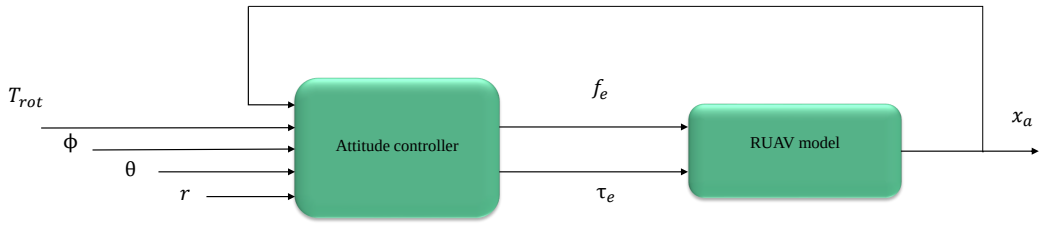


Figure 3.4: Attitude control.

where f_{a_c} are the expressions represented in Equation (3.1). To do this the forward Euler method was implemented to obtain a simple discretization of the non-linear system and the function takes a new form of

$$x_a(k+1) = f_a(x_a(k), u_a(k)) \tag{3.5}$$

with

$$f_a(x_a(k), u_a(k)) = x_a(k) + T_s f_{a_c}(x_a(k), u_a(k)) \tag{3.6}$$

where T_s is the sampling time of the simulation.

3.2.2 Internal controller

3.2.2.1 Attitude control

An internal embedded controller, in accordance with the paper used ([35]), was created so that the necessary external forces and torques can be produced. To do so the following equations were considered:

$$\begin{aligned} f_{ez} &= K_z T_{rot} \\ \tau_{ex} &= I_{xx}(K_\phi(\phi_{des} - \phi_a) - K_p p_a) - (I_{yy} - I_{zz}) q_a r_a \\ \tau_{ey} &= I_{yy}(K_\theta(\theta_{des} - \theta_a) - K_q q_a) - (I_{zz} - I_{xx}) p_a r_a \\ \tau_{ez} &= I_{zz} K_r (r_{des} - r_a) - (I_{xx} - I_{yy}) p_a q_a \end{aligned} \tag{3.7}$$

This simple controller receives as inputs the desired propellers thrust (T_{rot}), roll (ϕ_{des}), pitch (θ_{des}) and yaw rate (r_{des}) and, along with the proper gains for its inputs ($K_z, K_\phi, K_\theta, K_r$), produces the necessary force and torque for the drone to achieve the wanted attitude. Therefore, the new RUAV model state and input vectors can be defined as

$$x_a = \begin{bmatrix} P_a & \Phi_a & V_a & \omega_{Ba} \end{bmatrix}^T \qquad u_a = \begin{bmatrix} f_{ez} & \tau_{ex} & \tau_{ey} & \tau_{ez} \end{bmatrix}^T. \tag{3.8}$$

Figure 3.4 shows the attitude controller. The simulation of this controller will be performed further into this chapter.

3.2.2.2 Non-Linear internal controller

After obtaining results for the attitude control, a complete controller, was developed and implemented based on [42] and [43]. While the first controller uses the Euler angles directly in its equations, the non-linear internal controller, which will be dubbed Mellinger controller for the rest of this dissertation due to its origin, uses rotational matrices and errors, making the control more elaborate and versatile. The objective for this controller is to have it follow a position and velocity reference. For that an error between the reference and the current value of the parameter is created so that the controller works for the error to be zero. The position and velocity errors can be defined as

$$\begin{aligned} e_{pos} &= P_a - P_{a_{ref}} \\ e_{vel} &= V_a - V_{a_{ref}} \end{aligned} \tag{3.9}$$

where e_{pos} is the position error and e_{vel} is the velocity error. With this the desired force (F_{des}) can be obtained

$$F_{des} = -K_{pos}e_{pos} - K_{vel}e_{vel} + mgz_i + ma_{ref} \tag{3.10}$$

where K_{pos}, K_{vel} are positive definite diagonal matrices for the position and velocity controller gains, and z_i is the z-axis in the inertial frame. The thrust can be then obtained as

$$T_{rot} = F_{des} \cdot z_B \tag{3.11}$$

where $z_B = R_{IB}z_i$, in other words, the z-axis in the body frame. Next, the desired rotation matrix must be attained. The first step is to get $z_{B_{des}}$ that is described by

$$z_{B_{des}} = \frac{F_{des}}{\|F_{des}\|}. \tag{3.12}$$

Considering the intermediate coordinate frame of the inertial frame after a desired yaw rotation ψ_{des}, $x_{C_{des}}$ is given by

$$x_{C_{des}} = \begin{bmatrix} \cos\psi_{des} \\ \sin\psi_{des} \\ 0 \end{bmatrix}. \tag{3.13}$$

We can now obtain $y_{B_{des}}$ which is described by

$$y_{B_{des}} = \frac{z_{B_{des}} \times x_{C_{des}}}{\|z_{B_{des}} \times x_{C_{des}}\|} \tag{3.14}$$

where $\|z_{B_{des}} \times x_{C_{des}}\| \neq 0$. Finally $x_{B_{des}}$ is

$$x_{B_{des}} = y_{B_{des}} \times z_{B_{des}}. \tag{3.15}$$

Having obtained these three components we can now build the desired rotation matrix as

$$R_{IB_{des}} = \begin{bmatrix} x_{B_{des}} & y_{B_{des}} & z_{B_{des}} \end{bmatrix}. \tag{3.16}$$

Next we can get the rotation error (e_{rot}) given by

$$e_{rot} = \frac{1}{2}(R'_{IB_{des}}R_{IB} - R'_{IB}R_{IB_{des}})^V \tag{3.17}$$

where V is the inverse of the skew matrix.

Lastly we will work on the desired angular velocity $\omega_{B_{des}}$. To do so it is necessary to manipulate the expression of the derivative of the acceleration

$$m\dot{a}_{ref} = \dot{T_{rot}}z_{B_{des}} + R_{IB}\omega_{B_{des}} \times T_{rot}z_{B_{des}}. \tag{3.18}$$

Taking into account that $\dot{T}_{rot} = z_B \cdot m\dot{a}$ we can define vector $h_{\omega_{des}}$ as

$$h_{\omega_{des}} = R_{IB}\omega_{B_{des}} \times T_{rot}z_{B_{des}} = \frac{m}{T_{rot}}(\dot{a}_{ref} - (z_{B_{des}} \cdot \dot{a}_{ref})z_{B_{des}}) \tag{3.19}$$

We can then describe $\omega_{B_{x,des}}$ and $\omega_{B_{y,des}}$ as

$$\omega_{B_{x,des}} = -h_{\omega_{des}} \cdot y_{B_{des}} \tag{3.20}$$

$$\omega_{B_{y,des}} = h_{\omega_{des}} \cdot x_{B_{des}} \tag{3.21}$$

To calculate $\omega_{B_{z,des}}$ Equation (3.22) was used.

$$\omega_{B_{z,des}} = \dot{\psi}z_i \cdot z_{B_{des}} \tag{3.22}$$

We can now get the angular velocity error (e_ω) given by:

$$e_\omega = \omega_{Ba} - \omega_{Ba_{des}} \tag{3.23}$$

The input for the torques can be calculated as

$$\tau_{des} = -K_{rot}e_{rot} - K_\omega e_\omega \tag{3.24}$$

where K_{rot}, K_ω are the rotation and angular velocity gains. What is left is to test the controller, adjusting the gains until we obtain a good response. Figure 3.5 shows the Mellinger controller divided into a position controller that calculates the external force and into an attitude controller which calculates the external torques.

3.2.3 Simplified RUAV model

As seen previously in this chapter the RUAV model is highly non-linear and has a significant complexity, which creates a need for high computational power. To circumvent this issue, and since the objective is to control the RUAV movements so it follows the other vehicle, the model for the NMPC was simplified. This is possible given that we have a non-linear controller that allows for the combined control of the complete system. This version of the RUAV model becomes

$$\dot{P}_a = V_a \tag{3.25}$$

$$\dot{\psi}_a = \omega_{a_z} \tag{3.26}$$

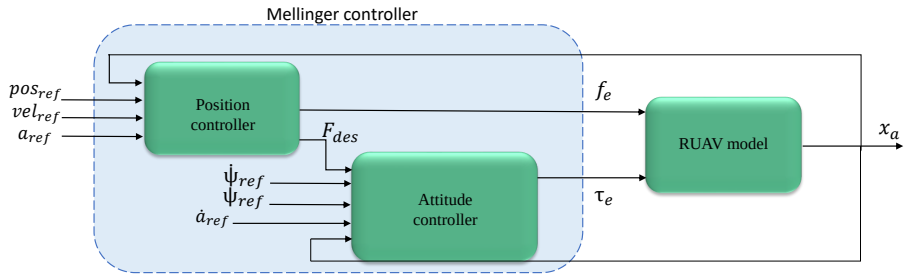


Figure 3.5: Mellinger controller

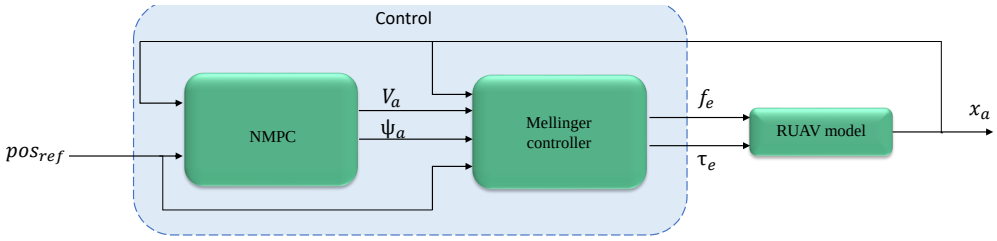


Figure 3.6: Simplified RUAV control.

where the states for the RUAV are now $x_a = \begin{bmatrix} P_a & \psi_a \end{bmatrix}^T$ and the inputs are $u_a = \begin{bmatrix} V_a & \omega_{a_z} \end{bmatrix}^T$. This is where the Mellinger controller comes in. Even though the NMPC has the ability to be applied to the controller of the full model, we started by implementing the simplified RUAV in the NMPC which then cannot control the full system model. By joining the two, feeding the outputs of the NMPC into the RUAV internal controller as references, it allows us to get a better grasp of the system, Figure 3.6 shows this combination.

For this model the same discretization process was applied except for the simplified version, f_{a_c} from Equation (3.6) is given by the expressions found in Equation (3.25).

3.3 Target Models

In this section the target vehicles will be modelled, which are the unmanned ground vehicle and two versions of the simplified model of a fixed-wing UAV, equivalent to the one created for the RUAV, since we only want to predict the general behaviour of these vehicles.

The following models, like the ones before, are continuous. The generalized model equation is given by

$$\dot{x}_i(t) = f_{i_c}(x_i(t), u_i(t)) \tag{3.27}$$

where f_{i_c} is the corresponding continuous model equations of the vehicle i, where f_{g_c} is for the ground vehicle and f_{fw_c} is for the fixed-wing UAV. As mentioned before the next

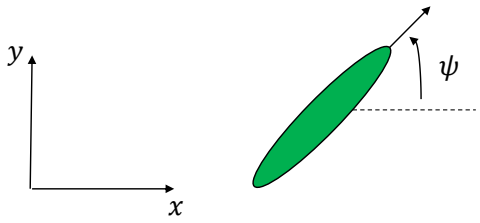


Figure 3.7: Generalized coordinates for a unicycle

step is to discretize the non-linear model which is now

$$x_i(k+1) = f_i(x_i(k), u_i(k)) \tag{3.28}$$

where f_i is obtained using the Euler forward method and is given by

$$f_i(x_i(k), u_i(k)) = x_i(k) + T_s f_{i_c}(x_i(k), u_i(k)) \tag{3.29}$$

3.3.1 Unmanned Ground vehicle

The UGV was modelled as a unicycle mobile robot according to [44]. A unicycle mobile robot is typically a robot which moves in a 2D world while having forward speed but no immediate lateral motion. Despite its name the unicycle robot doesn't necessarily have only one wheel, being typically used for two-wheel cars and sometimes four wheel vehicles. Modelling this system consists of studying the robot's kinematics and dynamics. Kinematics pertains to the trajectories the mobile robot follows when subject to commanding speeds. The dynamics take into account the forces and intrinsic frictions affecting the robot, thus complementing the kinematic model. The unicycle model is then the simplest kinematic model, where it is assumed that the robot can be approximated as one wheel [45],[46]. A simplified diagram of the unicycle coordinates for this type of model can be seen in Figure 3.7. The kinematic model can be described as

$$\begin{aligned} \dot{P}_g &= R_g(\psi) V_B \\ \dot{\psi}_g &= \omega_z. \end{aligned} \tag{3.30}$$

The states $P_g = \begin{bmatrix} x_g & y_g \end{bmatrix}^T$ are the position of the UGV in the x-y axis expressed in I and ψ_g is the orientation, while the inputs are $V_B = \begin{bmatrix} V_x & 0 \end{bmatrix}^T$, where V_x is the desired linear velocity in the x-axis, and the desired angular velocity, ω_z, in the z-axis expressed in the body frame (B_g). $R_g(\psi)$ is the rotation matrix for the x-y axis of the yaw angle, which is given by

$$R_g(\psi) = \begin{bmatrix} c\psi & -s\psi \\ s\psi & c\psi \end{bmatrix}. \tag{3.31}$$



Figure 3.8: Unmanned ground vehicle model

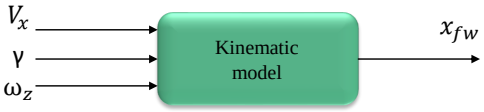


Figure 3.9: Fixed-wing UAV model.

The dynamic part was omitted, meaning that acceleration and forces are not considered, since the UGV has slower responses [35]. The advantage of using the unicycle robot is its simplicity and ability to model the most fundamental aspects of wheeled robots, in our case a simple ground vehicle that will later be complemented with a predictive controller. Figure 3.8 shows the UGV's model.

3.3.2 Fixed-wing UAV

In the kinematic sense a ground vehicle is very similar to a fixed-wing drone, since it describes the motion of the bodies without considering the forces that move them, except for the fact that a fixed-wing drone has movement in the z-axis. To add this coordinate, the flight-path angle (γ) is introduced in the inputs of the model to account for the vertical component of the vehicle displacement, which turned it into

$$\begin{aligned} \dot{P}_{fw} &= R_{fw}(\lambda)V_B \\ \dot{\psi}_{fw} &= \omega_z \end{aligned} \tag{3.32}$$

where the major difference lies in the position state which is now $P_{fw} = \begin{bmatrix} x_{fw} & y_{fw} & z_{fw} \end{bmatrix}$, the velocity vector, representing the components in the wind frame of the vehicle turns into $V_B = \begin{bmatrix} V_x & 0 & 0 \end{bmatrix}^T$, and in the rotation matrix between the wind frame and the inertial frame, that accounts for the yaw angle (ψ) and flight-path angle with $\lambda = \begin{bmatrix} \gamma & \psi \end{bmatrix}^T$, that can be described as

$$R_{fw}(\lambda) = \begin{bmatrix} c\psi c\gamma & -s(\psi) & c\psi s\gamma \\ s\psi c\gamma & c\psi & s\psi s\gamma \\ -s\gamma & 0 & c\gamma \end{bmatrix}. \tag{3.33}$$

The inputs that are now $u_{fw} = \begin{bmatrix} V_x & \gamma & \omega_z \end{bmatrix}^T$. Figure 3.9 shows the fixed-wing UAV model.

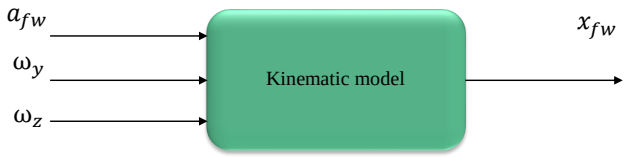


Figure 3.10: Fixed-wing UAV model with acceleration.

Table 3.1: Constant values for the attitude control

Parameter	Value
mass (kg)	2.1
I_{xx} $(kg.m^2)$	$3.8 * 10^{-3}$
I_{yy} $(kg.m^2)$	$3.8 * 10^{-3}$
I_{zz} $(kg.m^2)$	$7.1 * 10^{-3}$
a_T $(Ns.m^{-1})$	$4.8 * 10^{-2}$
a_R $(Ns.m^{-1})$	$6.4 * 10^{-4}$
g $(m.s^{-1})$	9.81

3.3.3 Fixed-wing UAV with acceleration

The next step in adding complexity to the fixed-wing UAV is to add linear acceleration to the model. The flight-path angle and the linear velocity (V_{fw}) were also converted into states rather than inputs, and the angular velocity in the y-axis (ω_y) and acceleration (a_{fw}) were added to the inputs. The model is then

$$\dot{P}_{fw} = R_{fw}(\lambda)V_{fw} \tag{3.34}$$

$$\dot{V}_{fw} = a_{fw} \tag{3.35}$$

$$\dot{\lambda}_{fw} = \omega_{fw} \tag{3.36}$$

where λ_{fw} is the one shown previously and $\omega_{fw} = \begin{bmatrix}\omega_y & \omega_z\end{bmatrix}^T$. The states are now $x_{fw} = \begin{bmatrix}P_{fw} & V_{fw} & \lambda_{fw}\end{bmatrix}^T$ and the inputs are $u_{fw} = \begin{bmatrix}a_{fw} & \omega_{fw}\end{bmatrix}^T$. Figure 3.10 shows the new model.

3.4 Simulation

3.4.1 Attitude control

For this simulation a pulse signal with an amplitude of $\frac{\pi}{6}$ rad and a period of 50 seconds was chosen, as well as a propeller thrust of 20 N and an angular velocity in z of zero. The values of the constants in Table 3.1 are the ones obtained in [47]. In an effort to make it easier to simulate the controller, the values for the aerodynamic components were considered zero. However, a valid assumption for low speed operation is to neglect the aerodynamic effects, which we assume for the results provided below. In this case

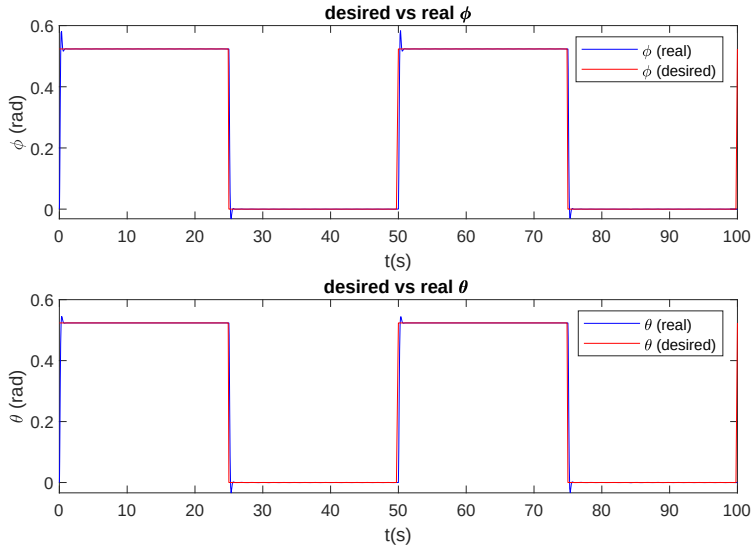


Figure 3.11: Plot of desired and obtained ψ and θ

Table 3.2: Values of the gains used in the attitude control

Parameter	Value
K_z	4
K_ϕ	170
K_θ	170
K_p	17
K_q	17
K_r	10

no constraints or limitations were applied to the control. In Figure 3.11 we can see the plot of ϕ_{des} and the actual ϕ, as well as for θ. We can see that the values obtained were very close to the desired ones. To obtain these outputs the values of the gains that were used can be found in Table 3.2. These gains were achieved through trial and error, by simulating the controller until the wanted results were obtained. Now that the attitude control loop has been elaborated we can start thinking about a more complete control loop, which will be done in the next section.

3.4.2 Non-Linear internal controller

This non-linear internal controller allows for us to give it a reference to follow and it would calculate the force and torques necessary to get there. For the next simulations, and until said otherwise, the x, y reference will be a circle with a radius of one meter, while z will be a constant also with the value of one meter. Having considered that the reference for velocity, ψ_{des}, $\dot{\psi}_{des}$, and $\dot{a}_{des}$ were all zero, in an effort to focus on position control, the gains obtained were the ones in Table 3.3, under the simplified column. This, of course is not realistic, but was done as a stepping stone to the full controller. The results of the states can be seen in Figure 3.12. After obtaining good results we moved on

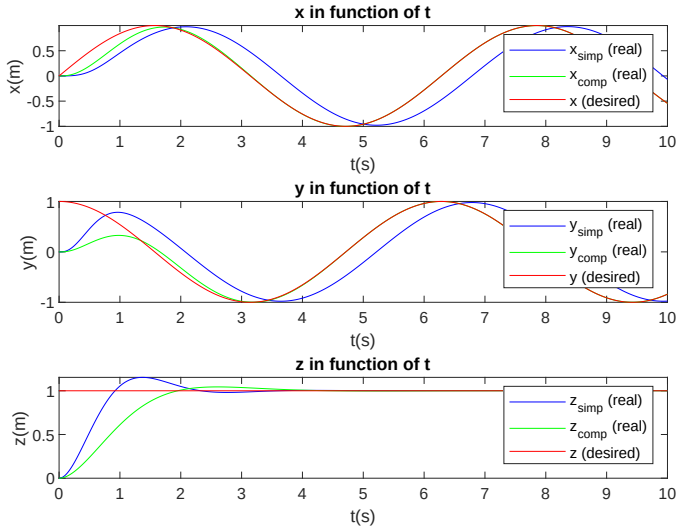


Figure 3.12: Position evolution of the simplified and complete RUAV Mellinger controller

Table 3.3: Table with the position controller gains.

	simplified	complete
K_{pos}	diag([8,8,7])	diag([8,8,6])
K_{vel}	diag([5,5,6])	diag([5,5,5])
K_{rot}	diag([8,8,8])	diag([8,8,8])
K_{ω}	diag([0.5,0.5,0.5])	diag([0.5,0.5,0.5])

to the development of the complete internal controller which is capable of calculating the inputs while considering all the the physical variables of the RUAV. Its gains can be seen in Table 3.3. Figure 3.13 shows the 3D plot for both versions of the controller. We can see that, although both versions are close to the reference, the full Mellinger controller not only reaches the intended target much faster, but it also stays much closer to it. In the simplified version we can see some overshoot in the z-axis as well as a considerable error between the reference and the vehicle.

3.4.3 UGV

After obtaining the model, the next step was moving to Simulink and obtaining some results for this vehicle. Since the UGV is a simple system and this thesis focuses on two aerial vehicles, an internal controller was not developed. In order to simulate the system two different signals were picked as inputs. For the linear velocity the signal is constant with a value of 5 $m.s^{-1}$. The signal for the angular velocity is a pulse wave with a period of 25 seconds and an amplitude of $\frac{\pi}{4}$ $rad.s^{-1}$. With a simulation stop time of 10 seconds the results obtained for the states can be seen in Figure 3.14. Figure 3.15 shows the ground vehicle movements in the x-y plane, as well as its orientation represented by blue arrows.

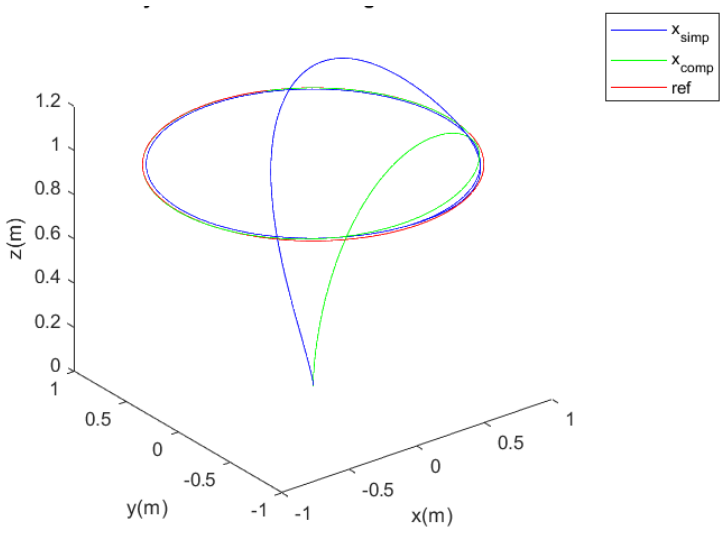


Figure 3.13: Plot for the 3D coordinates of the UAV with a Mellinger simplified and complete controller.

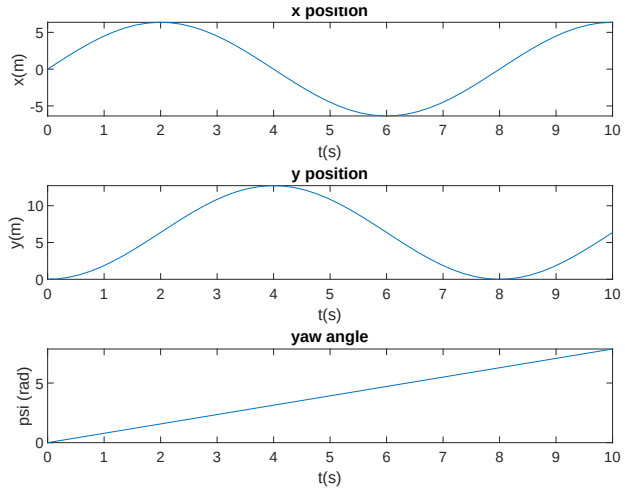


Figure 3.14: UGV state evolution

We can see that the UGV is able to move in a circle with the appropriate orientation. The addition of the NMPC will allow the vehicle to follow a position reference instead.

3.4.4 Connecting the vehicles

Before moving on to understanding and building the NMPC control, the output of the UGV was fed to the input of the RUAV in order to check the results, which can be seen in Figure 3.16. We can see that the complete version is overall much more precise than the simplified version, showing better results. Figures 3.17 and 3.18 shows the 2D and 3D movement plots for these vehicles.

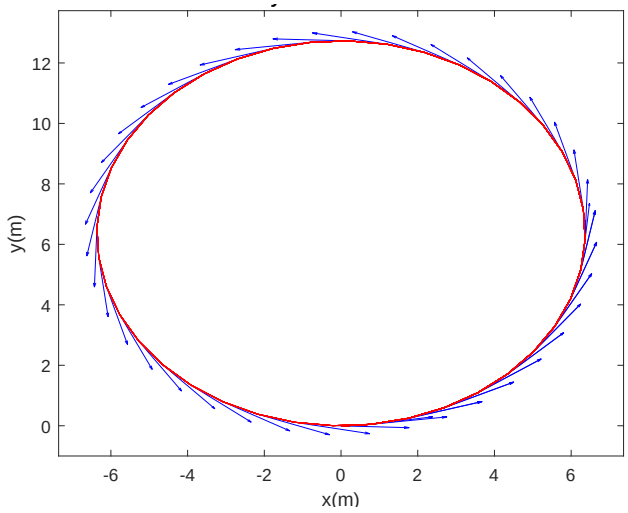


Figure 3.15: Movement of the UGV with y in function of x. Blue arrows represent the vehicles orientation

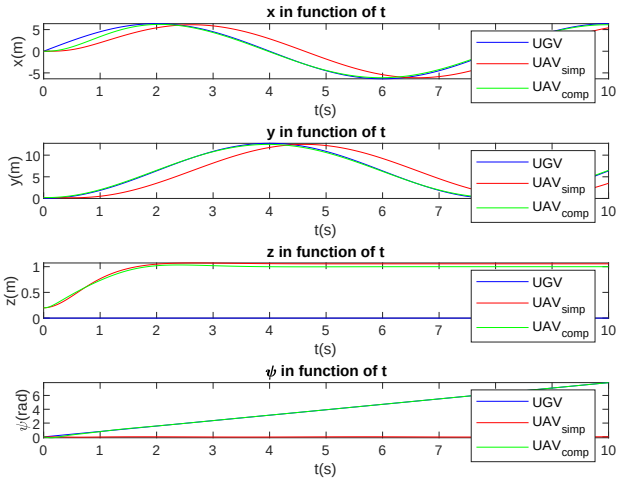


Figure 3.16: Plot for the UGV and RUAV state evolution of the simplified and complete controller

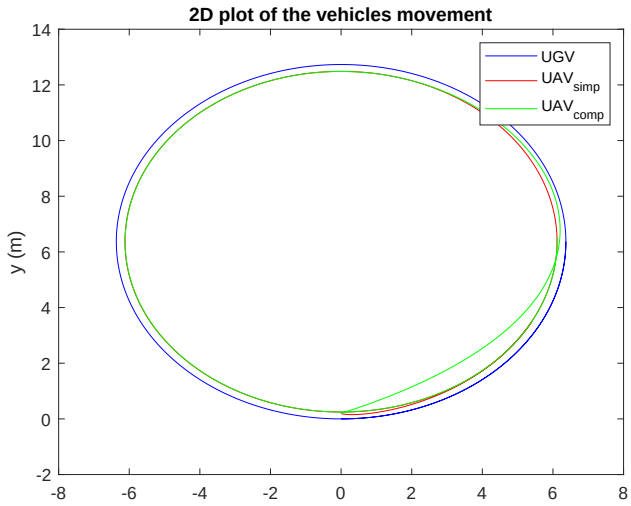


Figure 3.17: 2D plot for the UGV and RUAV, the simplified and complete controller

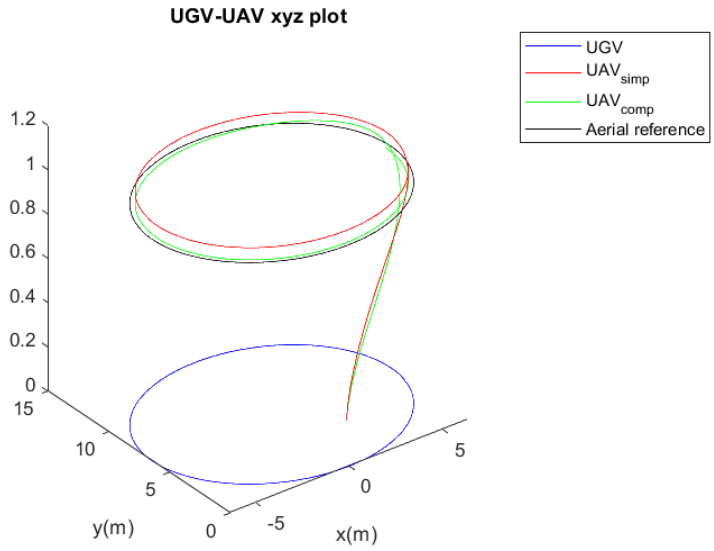


Figure 3.18: 3D plot for the UGV and RUAV, the simplified and complete controller

4

NMPC FOR JOINT TRAJECTORY TRACKING

In this chapter the NMPC control will be presented and developed. The main objective of the use of this type of control is for the vehicle to be able to regulate its position when given a reference to follow. Figure 4.1 shows what the intended final control for this section was. The process of building the NMPC for the target vehicles and the shuttle drone will be shown, as well as the simulation times for each vehicle. For both target vehicles the RUAV will not have the NMPC, using only its internal controller. However, when the NMPC for the RUAV is built the target vehicles already have and are using their non-linear controller.

4.1 Non-Linear control of target vehicles

The first thing to do is to describe the discrete non-linear control system as differential equations, as shown in Equation (3.28). One important aspect of the NMPC is that the current initial condition of the system is the previous state values, meaning $x_0(k) = x(k-1)$, hence moving the horizon forward.

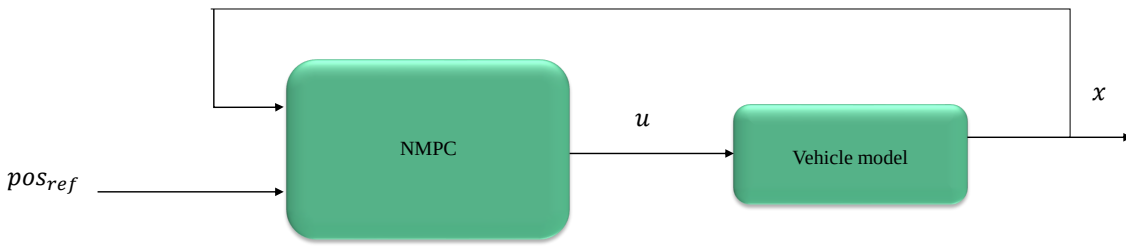


Figure 4.1: NMPC control

Table 4.1: NMPC - UGV gains

Q_g	diag([85,85,20])
P_g	$10Q_g$
R_g	diag([0.03,0.3])

4.1.1 Ground vehicle

For the ground vehicle the equality constraint is the same as Equation (3.30),

$$\begin{aligned}\dot{P}_g &= R_g(\psi)V_B \\ \dot{\psi}_g &= \omega_z.\end{aligned}$$

The states and inputs for the NMPC are the same as for the system model where

$$\begin{aligned}x_g &= \begin{bmatrix} x_g & y_g & \psi_g \end{bmatrix}^T \\ u_g &= \begin{bmatrix} V_x & \omega_z \end{bmatrix}^T.\end{aligned} \tag{4.1}$$

For all the NMPC controllers in this dissertation, it was considered that there was access to the states and inputs of the reference it was following. In this case the reference was a function created to draw a circle with a radius of one and a center at the x-y coordinates of zero. Because of this knowledge by the controller, it is able to predict the references next values, and then works on obtaining optimized state control sequence in order to drive the system along the wanted trajectory. To do this the specified performance index J_g, must be minimized relative to the given reference output signal. The cost for the ground vehicle is given by

$$\begin{aligned}\min_{u_{g_0},\dots,u_{g_{N-1}}} \quad & J_g = (y_{g_N} - \bar{y}_{g_N})^T P_g (y_N - \bar{y}_{g_N}) + \sum_{K=0}^{N-1} (y_{g_k} - \bar{y}_{g_k})^T Q_g (y_k - \bar{y}_{g_k}) + u_{g_k}^T R_g u_{g_k} \\ s.t \quad & x_{g_{k+1}} = f_{g_c}(x_{g_k}, u_{g_k}), \forall k = 0,\dots,N-1 \\ & y_{g_k} = C_g x_{g_k} \\ & x_{g_0} = z_{g_0}\end{aligned} \tag{4.2}$$

which can then be described in the batch approach as explained in Chapter 2. Another aspect of model predictive control is the input and state constraints that were described in Equations (3.28) and (3.29). After defining the cost function and the state and input constraints, we can execute Algorithm 1. To minimize the cost function over the prediction horizon the Matlab function `fmincon` was used. For the UGV the sampling time picked was of 0.1 seconds with a horizon of three. The output matrix (C_g) is a three-by-three identity matrix, since all the states are being controlled. The values of the gains for this controller can be found in Table 4.1.

These are the elements that then form the cost matrices, as mentioned before. Figure 4.2 shows the state evolution for the ground vehicle, as well as the reference. We can see that the NMPC controller works very well at predicting where the reference will be

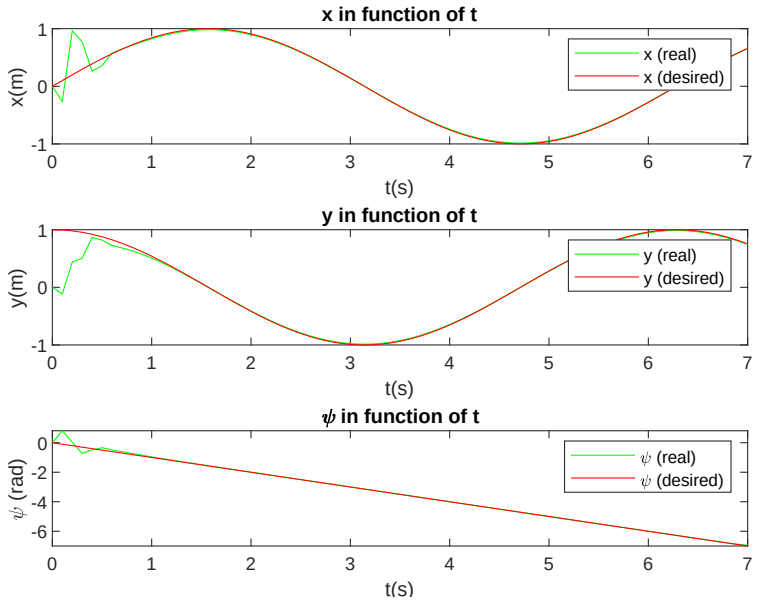


Figure 4.2: State evolution for the ground vehicle NMPC.

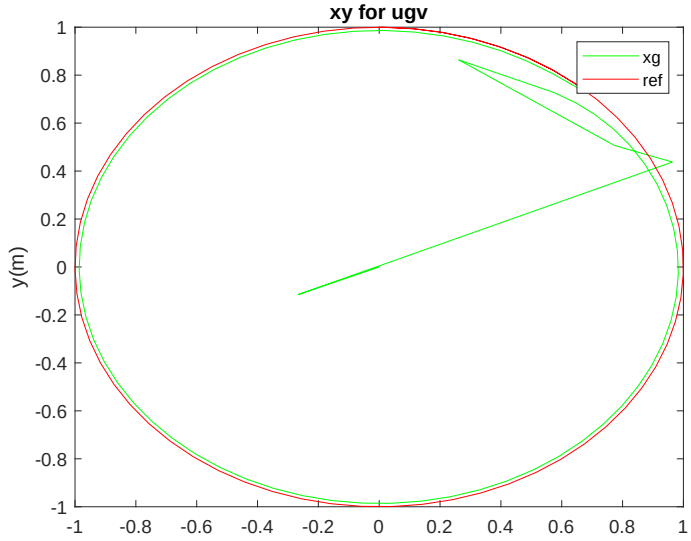


Figure 4.3: x-y plane for the ground vehicle NMPC

and obtaining the necessary inputs so that the system model can follow the reference. It is also of note that it takes the system around 0.4 seconds to stabilize and follow the reference smoothly, but after that there aren't any issues. Figure 4.3 shows the plot of the x-y plane, where we can see that the ground vehicle doesn't immediately follow the reference, taking some time to calibrate. Since the reference is always changing and we don't have integral or feed-forward control, we have a static error. As seen before in the state evolution after a bit that problem is solved, and the ground vehicle stays very close to the desired goal.

Figure 4.4 shows the input evolution for the NMPC. In this plot, we can see that those 0.4 seconds are the most active in terms of input, and after that the control action becomes

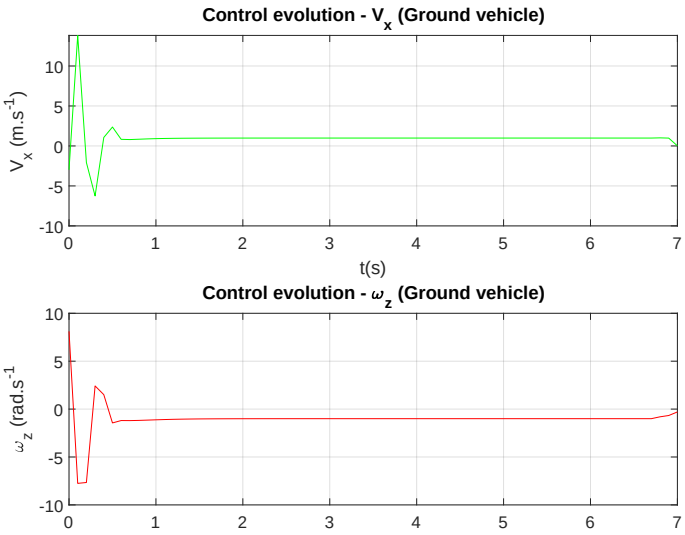


Figure 4.4: Input evolution for the NMPC (Ground vehicle)

much smoother and constant for both velocities. We also see that the constant is different from zero since both the position and the orientation change to follow the circular reference. This comes from the fact that the difference of position of the reference is very small from one instant to the other. After that initial control, the UGV starts following the reference closely, and the intensity of control becomes much smaller, applying only what is needed to maintain course. An important aspect of control is the computational weight of any given algorithm and the time it takes for it to be applied. Table 4.4, shows the overall run-time of the simulink simulation, while Figure 4.5 shows the time of each iteration of the algorithm. We can see that the first six iterations take up the most time, and that after that the time they take stabilizes, with some iterations taking more than 0.1 seconds to calculate. This shows that Simulink waits for the NMPC to finish calculating before moving on. In a real life scenario, more efficient libraries will be used to correct this. There is also some discrepancy between the sum of all the iterations and the run-time of the complete simulation, this can be explained by the initialization of the parameters and by the other controllers and models that are being run alongside the NMPC.

4.1.2 Fixed-wing UAV

The NMPC for this vehicle was developed very similarly to the one in the previous section. The equality constraint $f_{c_{fw}}$, can be found in Equation (3.34),

$$\dot{P}_{fw} = R_{fw}(\lambda)V_B$$
$$\dot{\psi}_{fw} = \omega_z$$

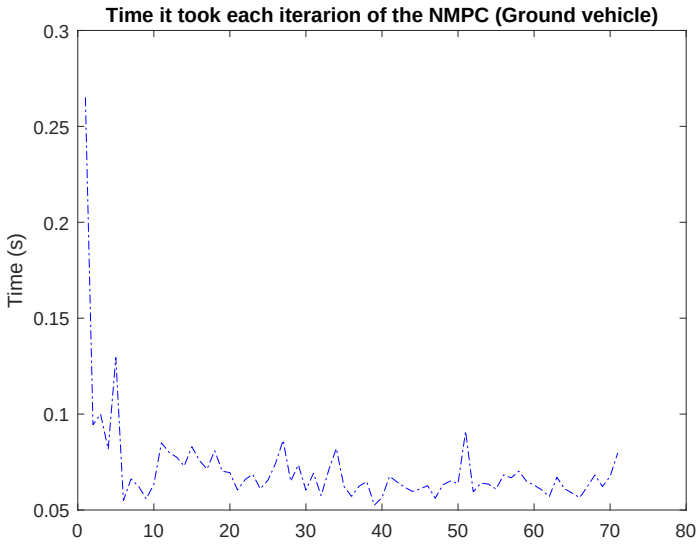


Figure 4.5: Time of each iteration for the UGV NMPC algorithm

Table 4.2: NMPC - fixed-wing UAV gains for $T_s = 0.1$s and $N = 15$

	FW UAV
Q	diag([455,455,455,85,15,15,85,80])
P	$10Q_{fw}$
R	diag([0.0001,0.0001,0.001,0.001,0.01])

and the states are

$$\begin{aligned} x_{fw} &= \begin{bmatrix} P_{fw} & V_{fw} & \lambda_{fw} \end{bmatrix}^T \\ u_{fw} &= \begin{bmatrix} a_{fw} & \omega_{fw} \end{bmatrix}^T. \end{aligned} \tag{4.3}$$

In this case the sampling time and the horizon stayed the same, and the algorithm was applied to the fixed-wing case, only altering the content of the matrices which change depending on the number of inputs and states. Table 4.2 shows the values of the cost matrices. As we can see in Figure 4.6 the states that represent the position are following the reference given, with an error. The addition of the acceleration as an input adds an immediate smoothness to the control of this model, which will be even more prominent in the following chapters. For the linear velocity, as seen in Figure 4.7, the state in the x-axis doesn't follow the reference very well. This can be because the gain is not well adjusted or an inherent problem in the model applied. Although kinematics wise the model can represent a fixed-wing UAV, in terms of real-world dynamics it does not represent the vehicle. Finally, in Figure 4.8 we see the final two states, the flight-path and yaw angles. Like the position states, we can see that the model follows the reference closely and without much deviation. Figure 4.9 shows the input evolution for the acceleration. When compared to the acceleration of the reference we can see that it remains close to the desired value in the x-axis while the other axis are a bit more lacking. For the angular velocity we can see in Figure 4.10 that after a small period of big variations in the values,

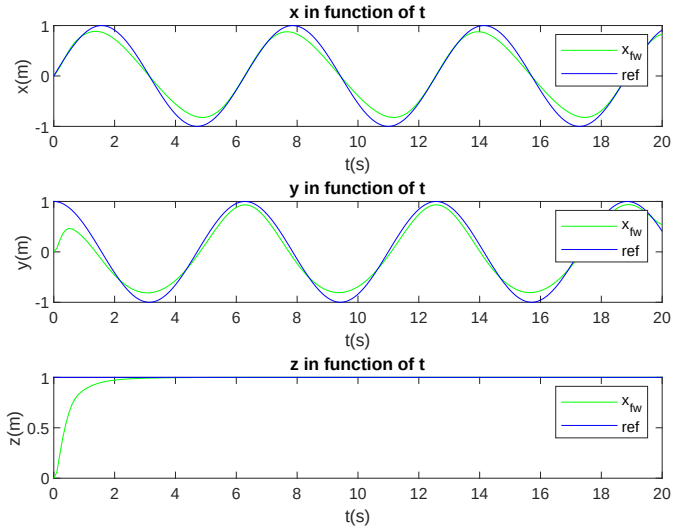


Figure 4.6: Position states for the fixed-wing UAV

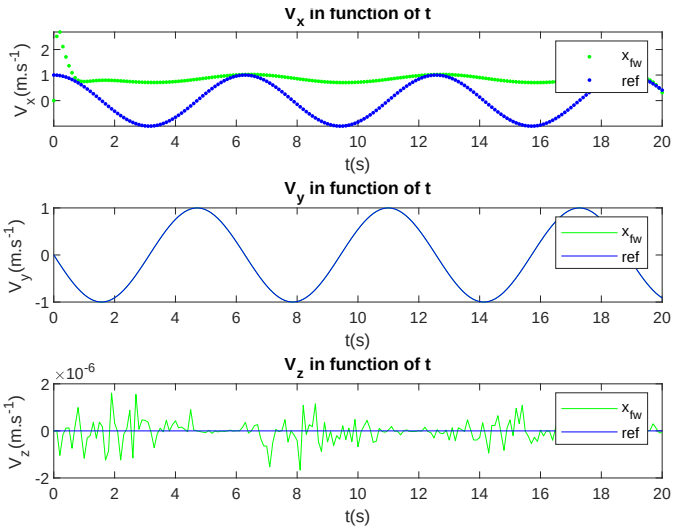


Figure 4.7: Linear velocity states for the fixed-wing UAV

which can be explained by the difference in positions at the start of the simulation, both velocities don't show big variations. Figure 4.11 shows the 3D plot of the simulation. We can see that the fixed-wing UAV runs smoothly throughout the simulation, however it clearly needs to be improved. Although the values come close to the desired ones there is still a big difference between the obtained results and the reference. Continuing the gain fitting is one of the options to improve this, as well as the building of a more realistic and complex model. Finally we will look at the time each iteration took in this simulation. The first thing we can glean from this plot is that the time that each iteration takes has grown substantially when compared to the UGV, which was expected since the model used is much more complex.

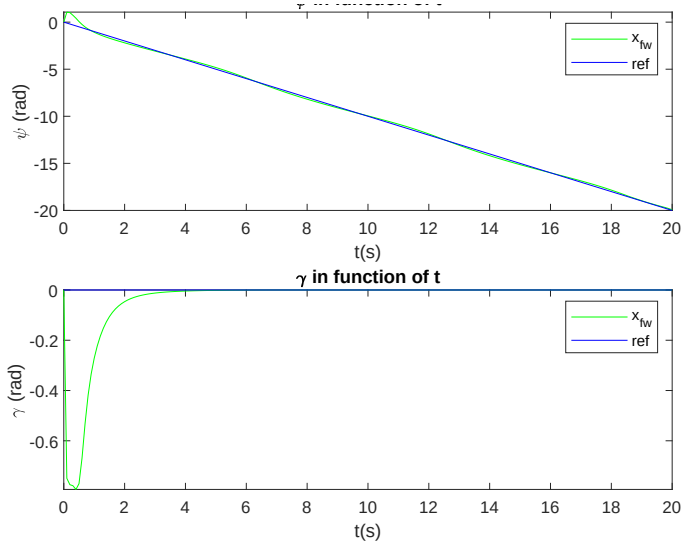


Figure 4.8: Angle states for the fixed-wing UAV

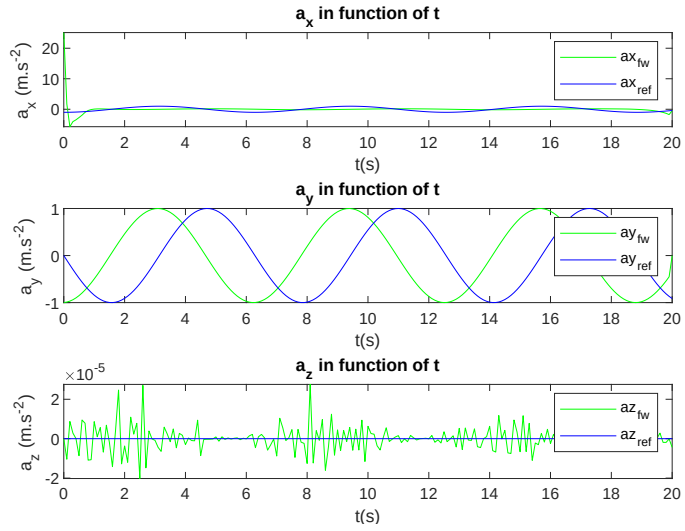


Figure 4.9: Input evolution of the acceleration for the fixed-wing UAV

4.2 Non-Linear control of the rotary UAV

After the UGV is complete and tested we can move on to the RUAV. From this section on both vehicles will have NMPC control, not just the target vehicles. Since the RUAV is a much more complex system we will use a simpler model of the RUAV, in the control as well as the system, in order to get to the differential game. This is done because the full model is much heavier computationally wise and takes much more time to test and simulate. While the ground vehicle and the simplified versions of the RUAV take less than eight minutes to simulate, the complete RUAV model took no less than thirty minutes, Table 4.4 shows the elapsed time of the simulations.

The process for the RUAV NMPC control is the same as the UGVs. The NMPC equality

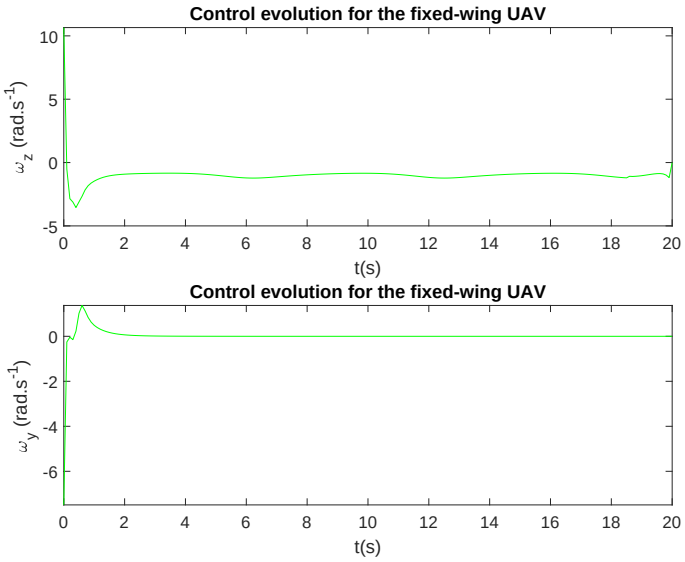


Figure 4.10: Input evolution of the angular velocity for the fixed-wing UAV

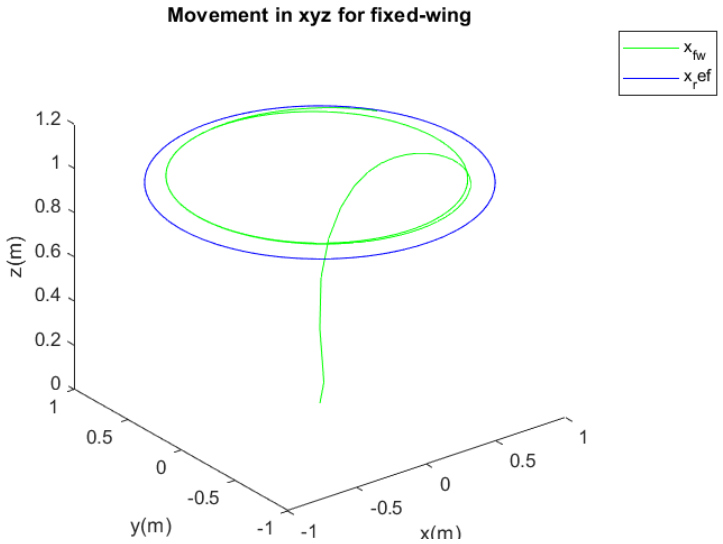


Figure 4.11: 3D plot of the position for the fixed-wing UAV

constraint f_{c_a}, and system model, will then be the one found in Equation (3.25),

$$\begin{aligned} \dot{P}_a &= V_a \\ \dot{\psi}_a &= \omega_{a_z} \end{aligned}$$

and the states and inputs are

$$\begin{aligned} x_a &= \begin{bmatrix} P_a & \psi_a \end{bmatrix}^T \\ u_a &= \begin{bmatrix} V_a & \omega_z \end{bmatrix}^T \end{aligned} \tag{4.4}$$

For this step, instead of using the Mellinger controller as seen in Figure 3.6, the system model was altered to match the model the NMPC is trying to control. The sampling time

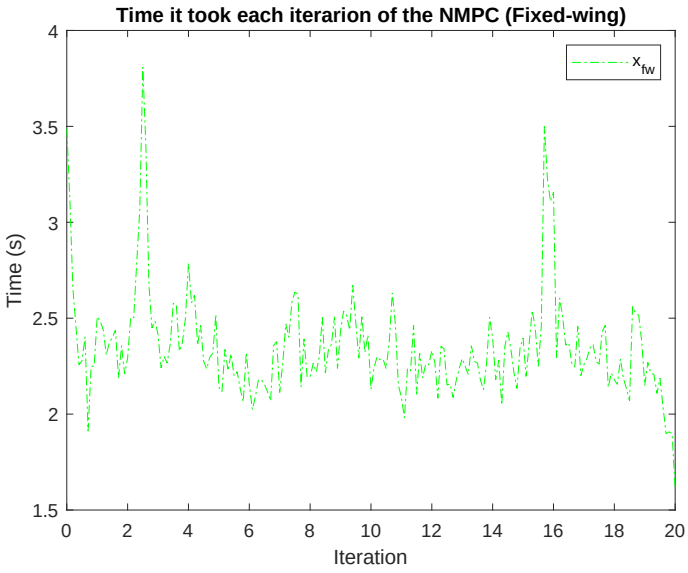


Figure 4.12: Time of each iteration for the NMPC fixed-wing UAV with acceleration

Table 4.3: NMPC - RUAV and UGV gains for $T_s = 0.1$s and $N = 15$

	RUAV	UGV
Q	diag([35,35,25,35])	diag([45,45,25])
P	$10Q_a$	$10Q_g$
R	diag([0.1,0.1,0.1,0.1])	diag([0.7,0.5])

for this version of the RUAV can still be 0.1 seconds, matching the UGV initial values. The receding horizon, on the other hand will increase to 15. The same algorithm is applied to the RUAV as it was in the UGV altering only the function, and therefore the constraints, and the size of the matrices, which depend on the number of states and inputs. The values for the gains are shown in Table 4.3. It is also important to note that since the horizon is 5 times larger than the UGV's initial one, the gains suffer some alterations from the ones in Table 4.1. For the RUAV the batch version of the cost is given by

$$\begin{aligned} \min_{U_a} \quad & J_a = (Y_a - \bar{Y}_a)^T \bar{Q}_a (Y_a - \bar{Y}_a) + U_a^T \bar{R}_a U_a \\ s.t \quad & F(X_a, U_a) = 0 \\ & Y_a = H_a X_a \end{aligned} \tag{4.5}$$

In Figure 4.13 we can see the state evolution for the RUAV with a NMPC controller where it follows the reference closely and without variation. Figure 4.14 shows the input evolution where we can see a smooth and more predictable input when compared to its equivalent in the fixed-wing UAV. The xy-axis plot for this simulation can be seen in Figure 4.15. This plot shows that the RUAV is able to follow the reference quickly and smoothly. Lastly, we can see the time it takes for each iteration in Figure 4.16, which overall is higher than the UGV but lower than a more composite model like the fixed-wing with acceleration.

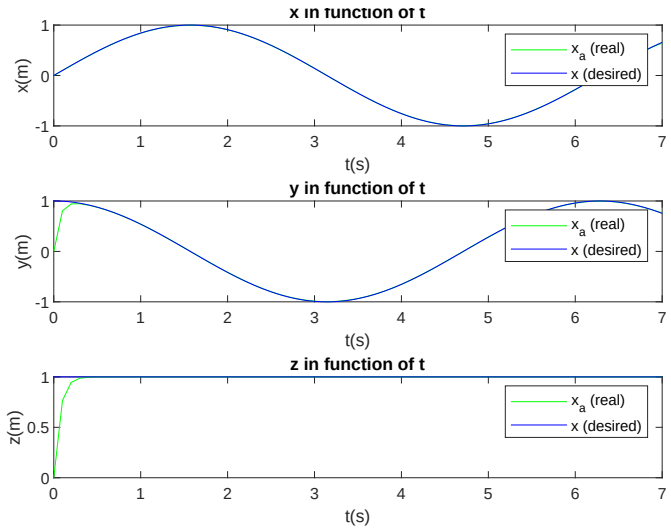


Figure 4.13: State evolution for the RUAV NMPC

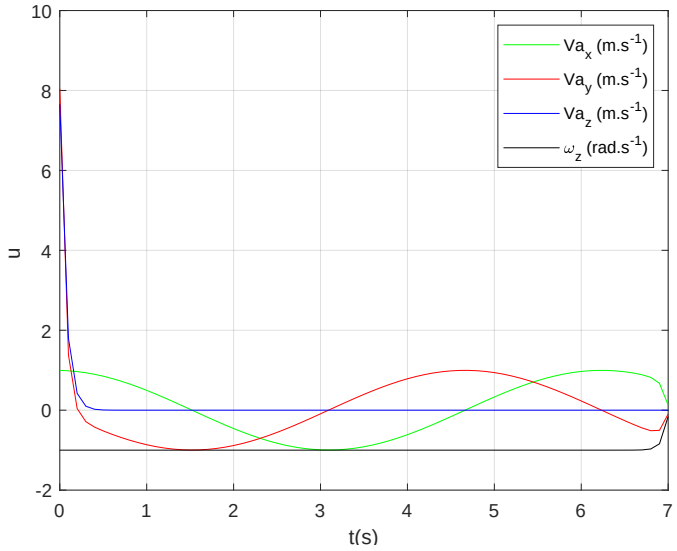


Figure 4.14: Input evolution for the RUAV NMPC

4.3 Joint tracking NMPC

In this section the results for both vehicles will be shown when the UGV and the shuttle drone are connected, and we have the target vehicle following the reference and the RUAV following the UGV.

There were two simulations for this NMPC. In one both the ground vehicle and the RUAV were following the same reference, while in the other the ground vehicle followed the reference and the RUAV followed the UGV. Figure 4.17 shows the state evolution for the position for both cases. We can see that the NMPC for the RUAV does a very good job in both cases, only being able to follow the reference marginally better than the UGV, since the latter has a less complicated model. It is also of note that when the RUAV is following the ground vehicle, the UGV is mostly one step ahead of the drone which is to

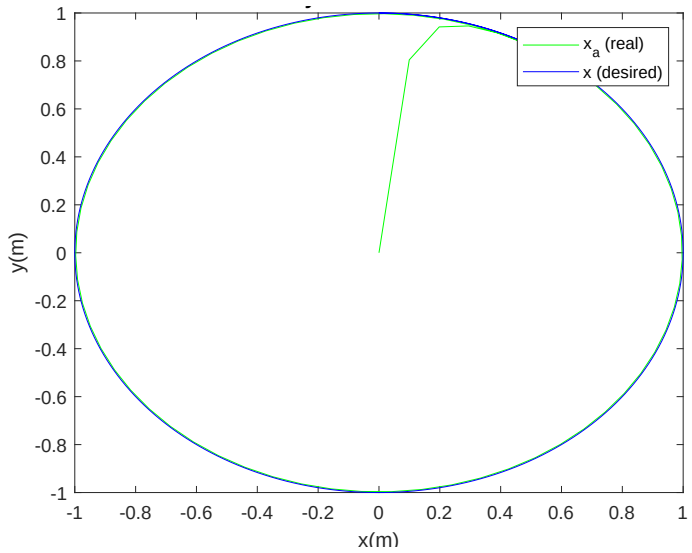


Figure 4.15: xy-axis plot for the RUAV NMPC

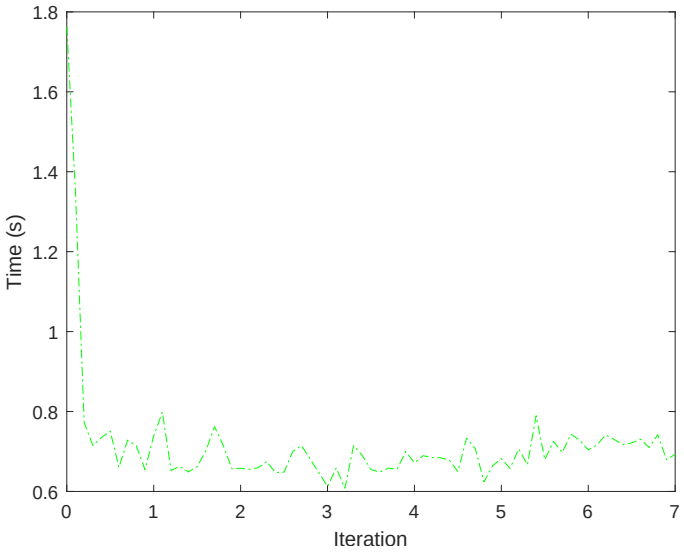


Figure 4.16: Time of each iteration for the RUAV NMPC

be expected. The same is not observed when both are following the reference, where the RUAV is quicker at getting to the desired goal.

For the RUAV most of the control changes between simulations, with the exception of the linear velocity in z that stays the same since the ground vehicle does not have a component in the z-axis, and the desired value for the RUAV is constant. Immediately we can see that when following the reference, the input for the RUAV is much smoother, this can be attributed to the fact that the changes in the reference itself are more fluid than the UGV. Figure 4.19 shows the movement of the vehicles in the x-y plane. We can see that the RUAV is faster and smoother when following the reference, and when following the UGV also stays very close to the desired value. In the plots of Figure 4.20 we can see that, overall, the time of each iteration of the RUAV is bigger than that of the UGV,

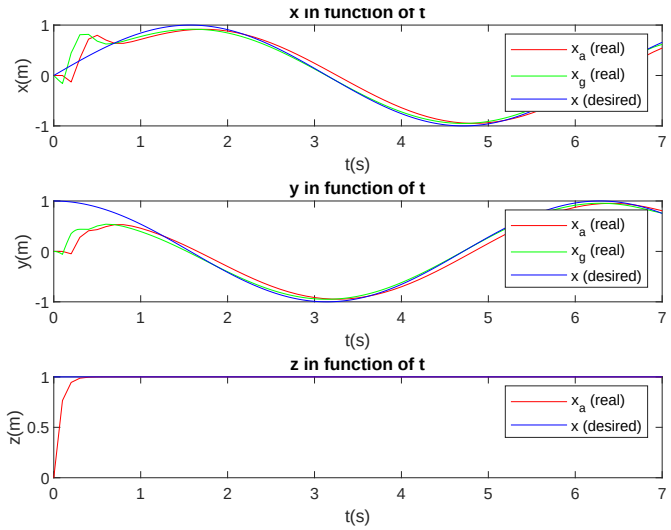


Figure 4.17: State evolution of the position states (connected NMPC RUAV and UGV).

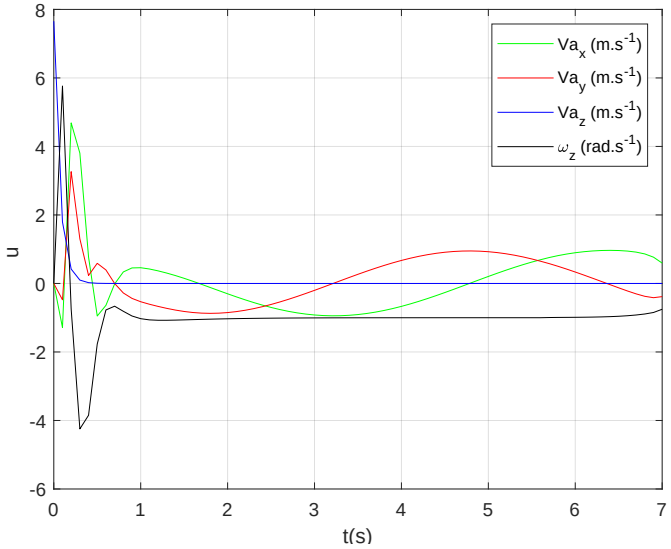


Figure 4.18: Control evolution for the RUAV connected to the UGV.

in both simulations. When adding all the iteration time we can see that the RUAV takes around twenty seconds. Since the RUAV has more states, and more inputs, than the UGV it makes sense that it takes extra time to calculate the most optimal input for the system model. Like before the sum of all the iteration times doesn't equal the run-time, since the iterations only count the time it takes the controller to calculate an optimal input value.

Finally, we will discuss the run-times for each case. The UGV simulation time is very short, staying under a minute. For the fixed-wing with acceleration the time jumps to an astounding 440 seconds. For the RUAV following the reference ($RUAV_{ref}$) the simulation takes twice as long, while when following the UGV ($RUAV_{con}$) the time difference is only around sixteen seconds. The run-time, when working with the full RUAV model,

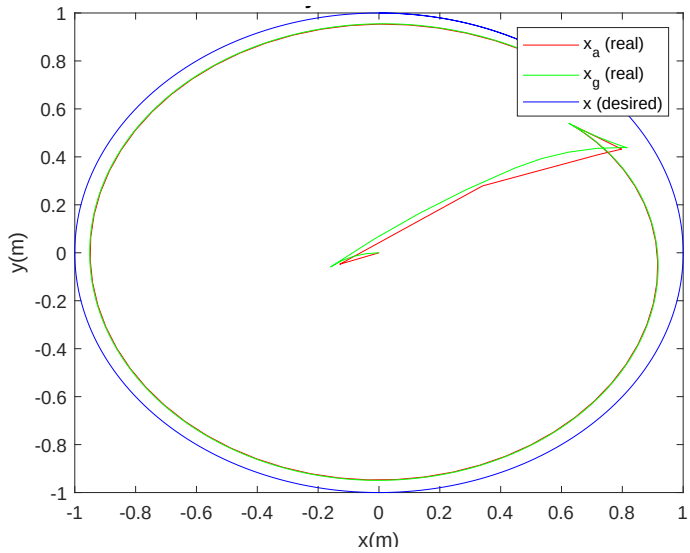


Figure 4.19: x-y plane of the vehicles with RUAV following UGV.

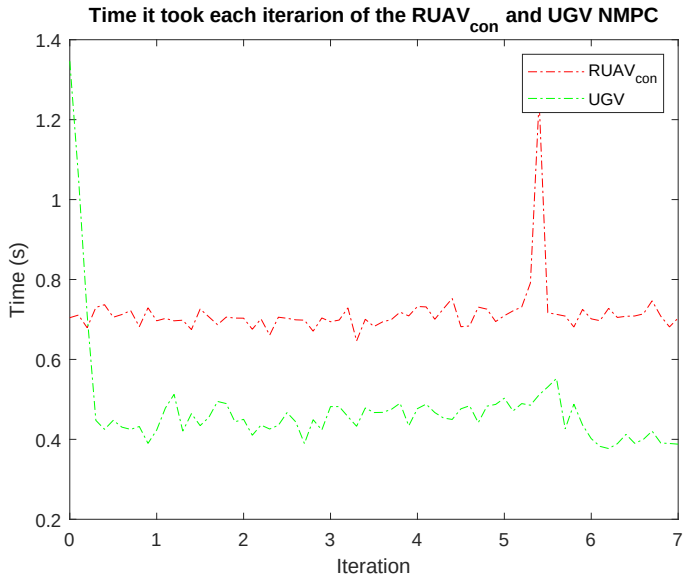


Figure 4.20: Plot for time in function of iteration for the connected UGV and RUAV

gets progressively higher, since the higher the computational weight, the bigger the runtime, as can be seen from the difference between the UGV and the fixed-wing UAV. Nevertheless, in the future it is important to develop the controller to its full potential, so that it can fully control the drone.

Table 4.4: Simulation times for each NMPC case

	Run time (s)	Sum of all iterations (s)
UGV	$\sim$ 58s	$\sim$ 8s
FW_{acc}	$\sim$ 441s	$\sim$ 439s
$RUAV_{ref}$	$\sim$ 136s	UGV = $\sim$ 43s | RUAV = $\sim$ 64s
$RUAV_{con}$	$\sim$ 120s	UGV = $\sim$ 44s | RUAV = $\sim$ 70s

5

NMPC-based Pursuit-Evasion Game

In this chapter we will describe the differential game created as well as show the resulting simulations. It is important to describe the optimization problem again since it changes slightly from the one described before. There are now two types of cost for the differential game. For the pursuer the specified performance index (J_p) must be minimized while considering a defined value for the control of the evader. In the case of the evader, the cost (J_e) must be maximized while considering a fixed control from the pursuer. The cost for the pursuer can be described as

$$\begin{aligned} \min_{u_{p_0},\dots,u_{p_{N-1}}} \quad & J_p = (y_{p_N} - y_{e_N})^T P_p (y_{p_N} - y_{e_N}) + \sum_{k=0}^{N-1} (y_{p_k} - y_{e_k})^T Q_p (y_{p_k} - y_{e_k}) + u_{p_k}^T R_p u_{p_k} \\ s.t \quad & x_{p_{k+1}} = f_{c_E}(x_{p_k}, x_{e_k}, u_{p_k}), \forall k = 0,\dots,N-1 \\ & y_{p_k} = C_p x_{p_k} \\ & x_{p_0} = z_{p_0} \end{aligned} \tag{5.1}$$

which in the batch formulation is

$$\begin{aligned} \min_{U_p} \quad & J_p = (Y_p - Y_e)^T \bar{Q}_p (Y_p - Y_e) + U_p^T \bar{R}_p U_p \\ s.t \quad & F_E(X_p, X_e, U_p) = 0 \\ & Y_p = H_p X_p. \end{aligned} \tag{5.2}$$

And the evader's cost can be described at

$$\begin{aligned} \max_{u_{e_0},\dots,u_{e_{N-1}}} \quad & J_e = (y_{e_N} - y_{p_N})^T P_e (y_{e_N} - y_{p_N}) + \sum_{k=0}^{N-1} (y_{e_k} - y_{p_k})^T Q_e (y_k - y_{p_k}) + u_{e_k}^T R_e u_{e_k} \\ s.t \quad & x_{e_{k+1}} = f_{c_E}(x_{p_k}, x_{e_k}, u_{e_k}), \forall k = 0,\dots,N-1 \\ & y_{e_k} = C_e x_{e_k} \\ & x_{e_0} = z_{e_0} \end{aligned} \tag{5.3}$$

Algorithm 2 Pursuer controller

1: **procedure** SOLVE MAXMIN J(E, U_p, U_e)
2: Maximize $J_e(E, U_p^*, U_e)$ to obtain U_e^*, considering the previously calculated U_p^*.
3: Minimize $J_p(E, U_p, U_e^*)$ to obtain U_p^*
4: Repeat the steps 2 and 3 k times, with k being the level of thinking
5: Set final U_p^* as the optimal input trajectory
6: **end procedure**
7: Set $u_p(1)$ as the current pursuer input

Algorithm 3 Evader controller

1: **procedure** SOLVE MINMAX J(E, U_p, U_e)
2: Minimize $J_p(E, U_p, U_e^*)$ to obtain U_p^*, considering the previously calculated U_e^*
3: Maximize $J_e(E, U_p^*, U_e)$ to obtain U_e^*
4: Repeat the steps 2 and 3 k times, with k being the level of thinking
5: Set final U_e^* as the optimal input trajectory
6: **end procedure**
7: Set $u_e(1)$ as the current evader input

which in the batch formulation is

$$\begin{aligned} \max_{U_e} \quad & J_e = (Y_e - Y_p)^T \bar{Q}_e (Y_e - Y_p) + U_e^T \bar{R}_e U_e \\ s.t \quad & F_E(X_p, X_e, U_e) = 0 \\ & Y_e = H_e X_e. \end{aligned} \tag{5.4}$$

For both cases the function for the equality constraints, F_E, can be defined by the stacking of both vehicles' equality constraints, or

$$F_E = \begin{bmatrix} F_p \\ F_e \end{bmatrix} \tag{5.5}$$

while the state for the pursuer is $X_p = \begin{bmatrix} x_p & x_e & u_p \end{bmatrix}^T$ and for the evader is $X_e = \begin{bmatrix} x_p & x_e & u_e \end{bmatrix}^T$. The main objective of the differential games is to have one player evade the other. To do so the evader and the pursuer have two different algorithms. Algorithm 2 shows the pursuers algorithm. Considering the previous input of the pursuer, it will calculate the best-case scenario of the evader, maximizing the cost shown in Equation (5.4). After obtaining the maximized input for the evader, the pursuer will then calculate its own optimal input, by minimizing the cost shown in Equation (5.2), it will then repeat these steps the same number of times as the horizon. The final step is to apply the first control action from U_p^* to the system. Algorithm 3 shows the evaders algorithm. The evaders algorithm is very similar to the pursuers. The main difference is that it first minimizes the pursuer's cost, obtaining the best-case scenario for the pursuer, and then uses the control values obtained to maximize its cost, in order to obtain the optimal control sequence. To solve these algorithms `fmincon` was once again used, this time each controller runs it twice, once to minimize the pursuer, and once to maximize the evader, and vice-versa. In

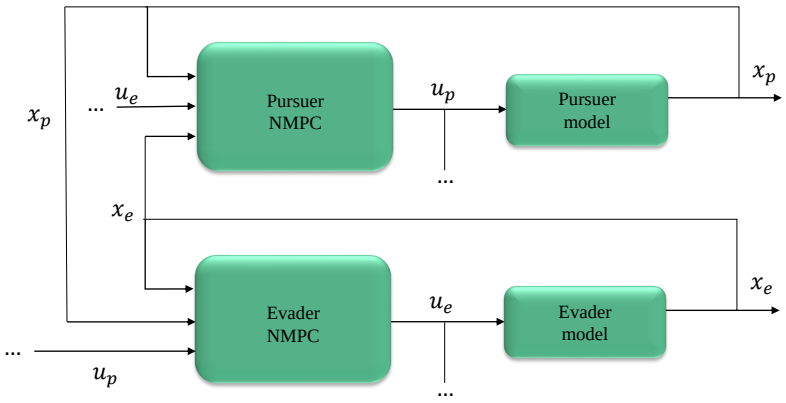


Figure 5.1: Schematic for the differential game between heterogeneous vehicles

Table 5.1: Cost matrices values for the UGV and RUAV differential game

	UGV	RUAV
Q	diag([75,75,75])	diag(35,35,35,35)
P	10*Q	10*Q
R	$s_g$diag([0.1,0.1])	$s_a$diag([0.1,0.1,0.1,0.1])

this case the players are dependent on each other, meaning that it was considered that they are both aware of their adversary's previous optimal input. In future works it is important to develop the game so that the players are independent. Figure 5.1 shows the schematic for the differential games shown in this chapter. The next chapters expand on these differential games in three different scenarios. In one scenario the evader is the ground vehicle, in the other the evader is the fixed-wing drone, and the last with a more developed fixed-wing evader. In all cases the pursuer is the rotary UAV, and in order to know when the evader has been caught, the RUAV only has to come within 0.05 meters of the other vehicle in all three position axis (XYZ). For these scenarios the receding horizon is once again 15 and the sampling time is 0.1 seconds.

5.1 Shuttle drone vs. Ground Target

We will start by considering the ground vehicle as the evader. Here $x_p = x_a$ and $x_e = x_g$. The constraint equality is given by $f_{c_e} = \begin{bmatrix} f_{c_a} & f_{c_g} \end{bmatrix}^T$ where the individual equality constraints are from Equations (3.25) and (3.30) respectively. The values of the matrices shown in the cost function are in Table 5.1, where s_a and s_g, respectively, for the RUAV and UGV, is the weight we give to the input in the cost function. The smaller the value of the input matrix the faster the vehicle will be since this means that the cost of the given input is smaller and therefore more of it can be given. For this case we will consider three scenarios:

Table 5.2: Input constraints for the differential game (UGV vs. RUAV)

		Input
Lower bounds	E.v	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [-10 & -10 & -10 & -2\pi]^T \\ [-10 & -10 & -10 & -2\pi]^T \end{matrix}$
	F.UGV	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [-10 & -10 & -10 & -2\pi]^T \\ [-12 & -12 & -12 & -2\pi]^T \end{matrix}$
	S.UGV	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [-12 & -12 & -12 & -2\pi]^T \\ [-10 & -10 & -10 & -2\pi]^T \end{matrix}$
Upper bounds	E.v	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [10 & 10 & 10 & 2\pi]^T \\ [10 & 10 & 10 & 2\pi]^T \end{matrix}$
	F.UGV	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [10 & 10 & 10 & 2\pi]^T \\ [12 & 12 & 12 & 2\pi]^T \end{matrix}$
	S.UGV	$\begin{matrix} RUAV \\ UGV \end{matrix} = \begin{matrix} [12 & 12 & 12 & 2\pi]^T \\ [10 & 10 & 10 & 2\pi]^T \end{matrix}$

1. Both vehicles have the same speed (E.v). ($s_a = s_g = 1$)

2. The ground vehicle is faster than the RUAV (F. UGV). ($s_a = 0.1$ and $s_g = 0.001$)

3. The RUAV is faster than the ground vehicle. (S.UGV) ($s_a = 0.001$ and $s_g = 0.1$)

In order to create these scenarios the input constraints are altered to fit the desired goal. Table 5.2 shows the values for the upper and lower bounds of the constraints. In Figures 5.2, 5.3, 5.4 we can see the values of the states along the simulation for all three scenarios. The starting point for the RUAV is $x_a = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}^T$ and for the UGV is $x_g = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$. We can see that in the first scenario, where the velocities are equal, as well as the weight of the inputs, the UGV is caught by the RUAV at 975 milliseconds. When the UGV is faster than the RUAV and $s_g = 0.001$, it is able to out maneuver the later. Lastly in the scenario where the RUAV has the advantage on the speed it can reach and the inputs weight less ($s_a = 0.001$), the drone catches the ground vehicle in 1.5 seconds.

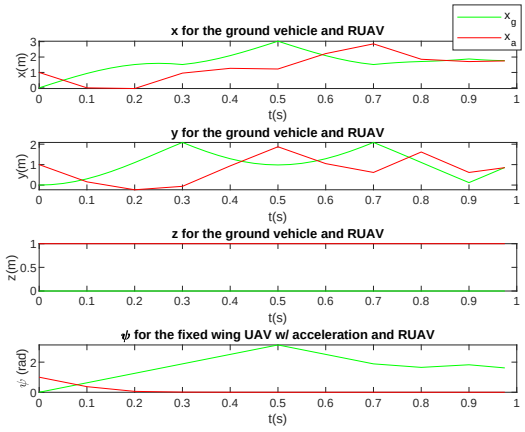


Figure 5.2: State evolution (UGV vs. RUAV) - Equal constraint

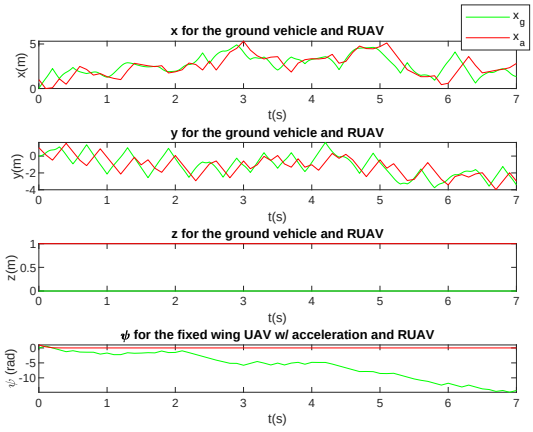


Figure 5.3: State evolution (UGV vs. RUAV) - Faster UGV

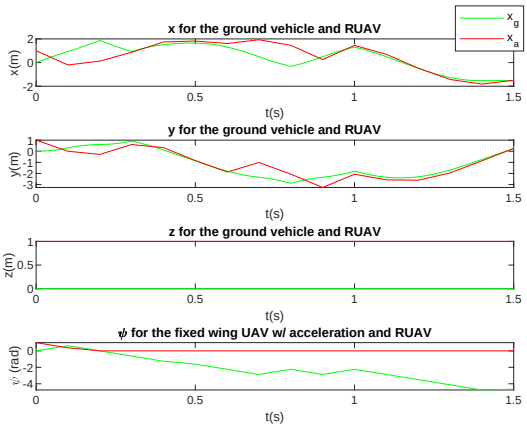


Figure 5.4: State evolution (UGV vs. RUAV) - Faster RUAV

Figures 5.5, 5.6, and 5.7 show the control evolution for the UGV and Figures 5.8, 5.9, and 5.10 show it for the RUAV. We can see that both inputs stay within their bounds.

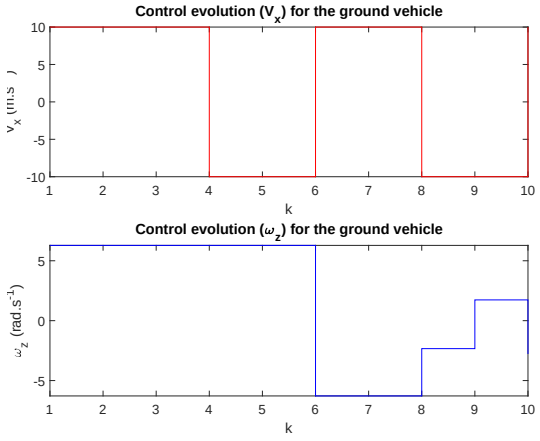


Figure 5.5: Input evolution (UGV) - Equal constraint

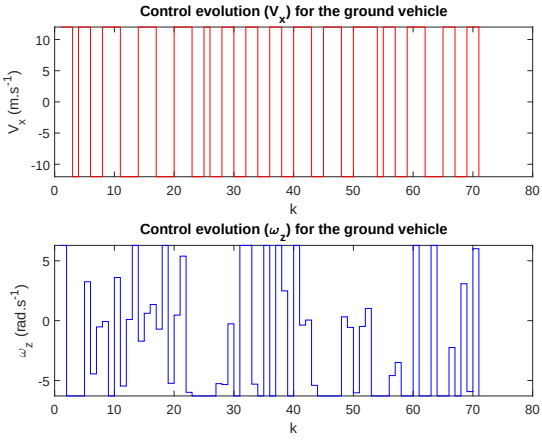


Figure 5.6: Input evolution (UGV) - Faster UGV

However the control is very erratic, instead of being smooth, one way of improving this is to develop more complex models for the NMPC. Furthermore we can see that it saturates many times which also causes the control to be less effective. A way around this would be to incorporate input variations (Δu), adding another integral effect. In Figures 5.11, 5.12, and 5.13 we can see the movement of the vehicles in the xyz plane. The turbulent movement is a direct reflection of the irregular control values. In this case, both vehicles follow a reference for the z-axis since in this scenario there is no need for the vehicles to follow each other in that axis. Once again we can see that overall the RUAV iterations take more time than the UGV's iterations. However the difference between the two diminishes greatly, since in both cases an `fmincon` is run for each vehicle. Finally the plot for the distance between each vehicle is shown. The results are obtained by using the following

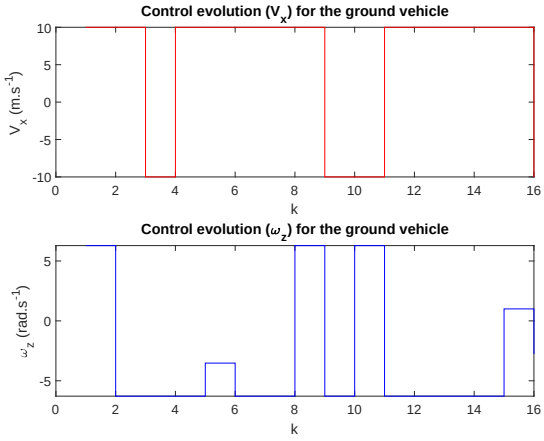


Figure 5.7: Input evolution of UGV - Faster RUAV

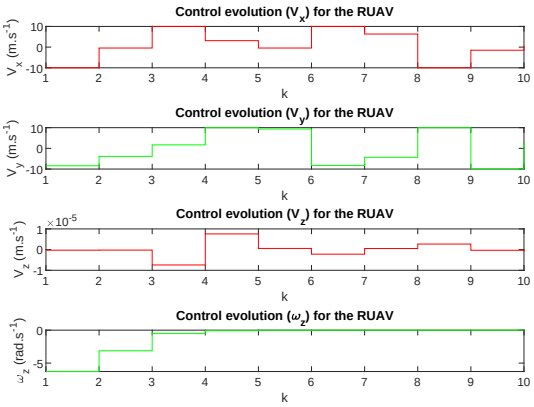


Figure 5.8: Input evolution of RUAV - Equal constraint

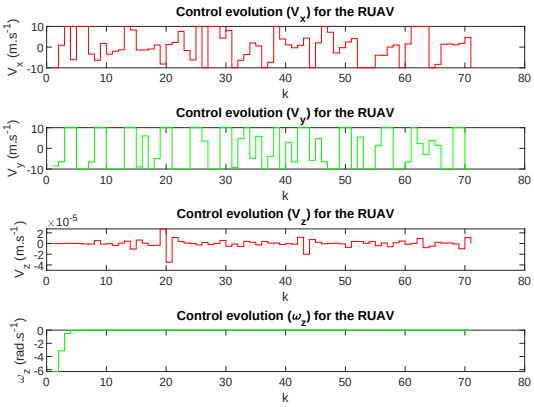


Figure 5.9: Input evolution of RUAV - Faster UGV

equation:

$$d = \sqrt{(x_a - x_g)^2 + (y_a - y_g)^2}. \tag{5.6}$$

We can see that when the RUAV gets very close to the the UGV it seems to stop its pursuit. This is given by the way the cost was developed. When the RUAV gets very close it slows down giving the UGV the opportunity to escape, realising that the distance has increased it starts pursuing again, until it catches the UGV in case one and three, or it manages to escape in case two.

5.2 Shuttle drone vs. Fixed-wing Target

In this case the pursuer and evader are $x_p = x_a$ and $x_e = x_{fw}$. The constraint equality is given by $f_{c_e} = \begin{bmatrix} f_{c_a} & f_{c_{fw}} \end{bmatrix}^T$ where the individual equality constraints are from Equations (3.25) and (3.32) respectively, where the control we mainly focus on is the velocity of the

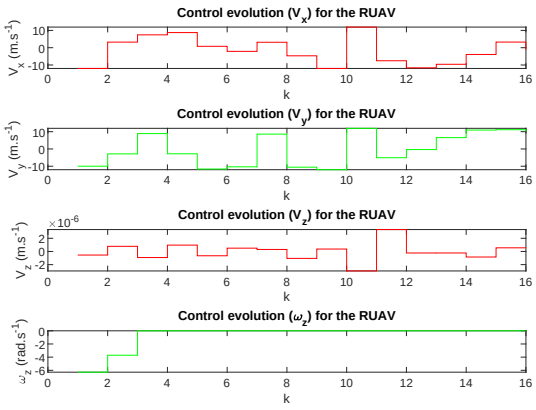


Figure 5.10: Input evolution of RUAV - Faster RUAV

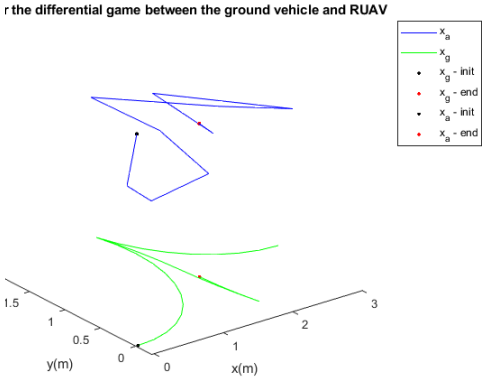


Figure 5.11: XYZ plot (UGV vs. RUAV) - Equal constraints

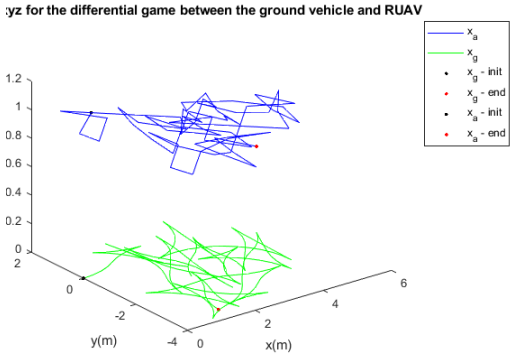


Figure 5.12: XYZ plot (UGV vs. RUAV) - Faster UGV

Table 5.3: Cost matrices values for the fixed-wing and RUAV differential game

	FW UAV	RUAV
Q	diag([75,75,35,75])	diag(35,35,35,35)
P	10*Q	10*Q
R	s_{fw}diag([0.1,0.1,0.01])	$s_a$diag([0.1,0.1,0.1,0.1])

vehicles. The values of the matrices shown in the cost function are in Table 5.3. where just like before s_i represents the weight of each vehicles input, with fw being for the fixed-wing UAV. For the constraints, the system remains the same, except now there is an extra input constraint for the fixed-wing UAV. Table 5.4 shows the values for this simulation. In this case the lower bound for the linear velocities of the fixed-wing UAV is fixed at 5 m.s^{-1}, since a plane drone shouldn't be able to fly straight backwards, this was not applied to the other vehicles. Similarly to the ground vehicle, the fixed-wing is not able to escape in scenario one, being caught close to six seconds. It is able to narrowly escape in scenario two, and in scenario three it is captured around the one second mark. Once again we can see that the input evolution is both erratic, particularly in the RUAV, and saturated, more

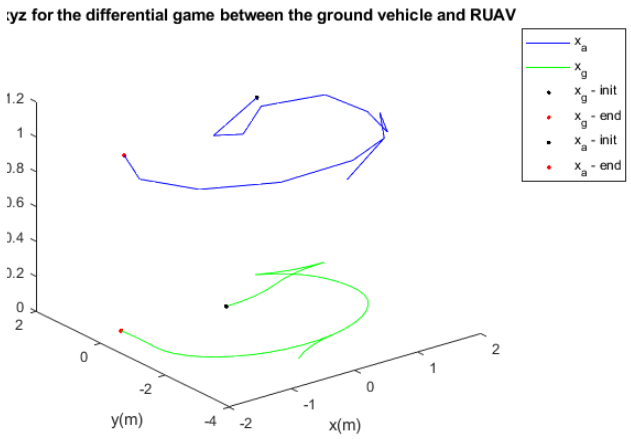


Figure 5.13: XYZ plot (UGV vs. RUAV) - Faster RUAV

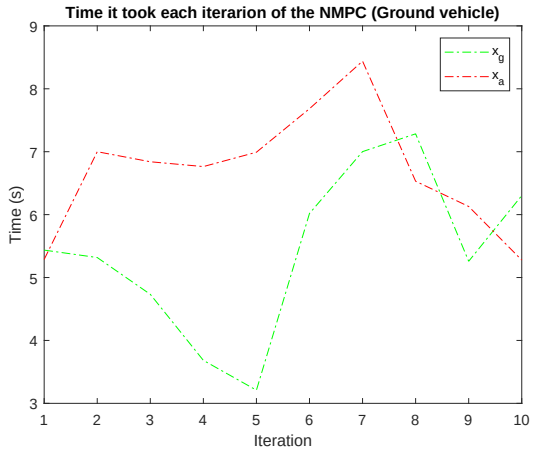


Figure 5.14: Time of each iteration (UGV vs. RUAV) - Equal constraints

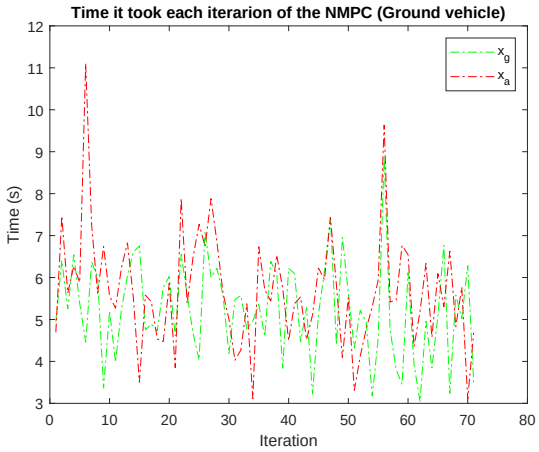


Figure 5.15: Time of each iteration (UGV vs. RUAV) - Faster UGV

on the fixed-wing UAV. In future works it is important to address this issue, using some of the ways shown before. Figures 5.29, 5.30, and 5.31 show the vehicles movements in the xyz plane. Here it is easier to see the pursuit, where the RUAV follows the fixed-wing drone very closely throughout the simulations. In this last scenario we can see that the time of each iteration gets much closer between the two vehicles, each of them passing the other at different times. This makes sense, considering that these vehicles models are much closer to each other, and have more states and inputs. Just like the UGV, the distance between each vehicle alternates, depending on what the RUAV is doing in the pursuit. This time it was calculated by

$$d = \|P_a - P_{fw}\|. \tag{5.7}$$

A way to prevent this is adding a fixed parameter to the cost function so that the RUAV stops permitting its target to escape once it gets close.

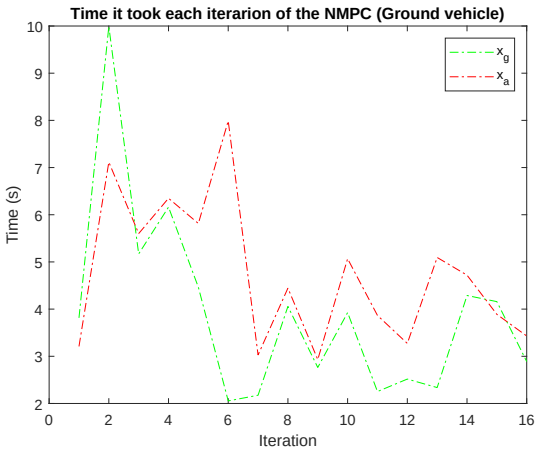


Figure 5.16: Time of each iteration (UGV vs. RUAV) - Faster RUAV

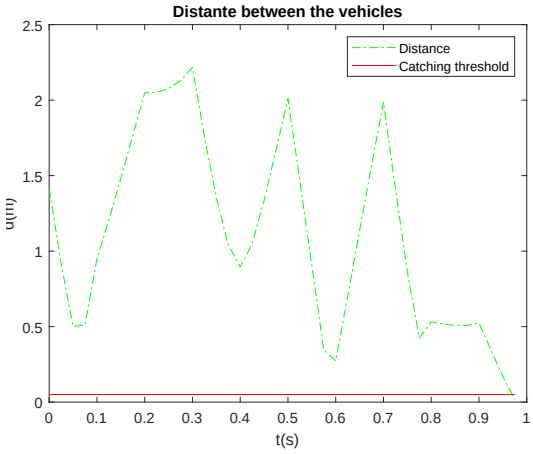


Figure 5.17: Distance between vehicles (UGV vs. RUAV) - Equal constraints

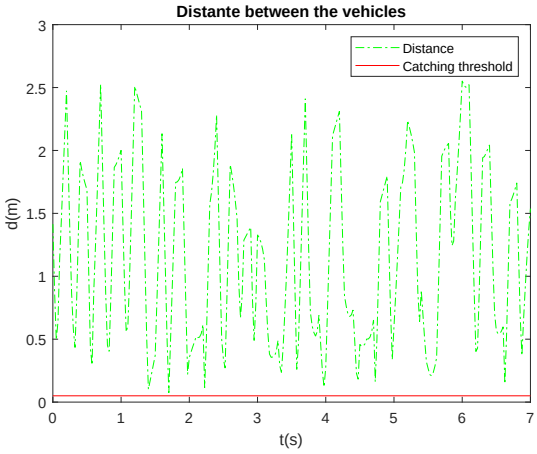


Figure 5.18: Distance between vehicles (UGV vs. RUAV) - Faster UGV

5.3 Shuttle drone vs. Fixed-wing Target with Acceleration Control

Applying the same steps as the previous section, we get the values for the cost matrices seen in Table 5.5. For this section the change was in the model which now accounts for acceleration as the input and the velocities as states, changing the equality constraint as well. Unlike the two other vehicles, we can no longer directly compare the two models since we made the fixed-wing UAV more complex. So, in this case there will be only two tests, one where the fixed-wing has the advantage, and one where the RUAV has the advantage, which will be done by changing the RUAV's velocity bounds. The output matrix, which until now was the identity matrix, changes so that only the four states are

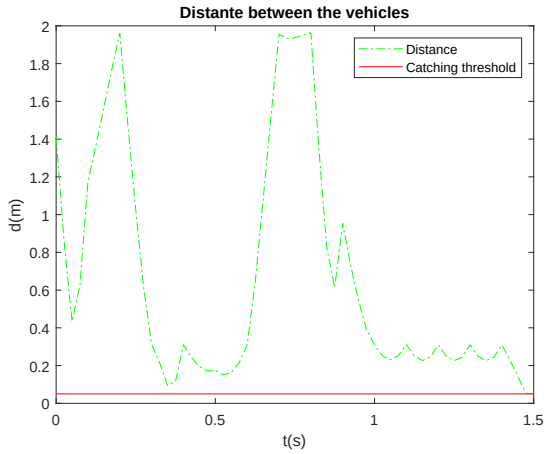


Figure 5.19: Distance between vehicles (UGV vs. RUAV) - Faster RUAV

Table 5.4: Input constraints (fw UAV vs. RUAV)

Lower bounds	E.v	$\underset{FW}{RUAV} = \begin{matrix}[-10 & -10 & -10 & -2\pi]^T \\ \left[5 & -\frac{4\pi}{9} & -2\pi\right]^T\end{matrix}$
	F.FW	$\underset{FW}{RUAV} = \begin{matrix}[-10 & -10 & -10 & -2\pi]^T \\ \left[5 & -\frac{4\pi}{9} & -2\pi\right]^T\end{matrix}$
	S.FW	$\underset{FW}{RUAV} = \begin{matrix}[-12 & -12 & -12 & -2\pi]^T \\ \left[5 & -\frac{4\pi}{9} & -2\pi\right]^T\end{matrix}$
Upper bounds	E.v	$\underset{FW}{RUAV} = \begin{matrix}[10 & 10 & 10 & 2\pi]^T \\ \left[10 & \frac{4\pi}{9} & 2\pi\right]^T\end{matrix}$
	F.FW	$\underset{FW}{RUAV} = \begin{matrix}[10 & 10 & 10 & 2\pi]^T \\ \left[16 & \frac{4\pi}{9} & 2\pi\right]^T\end{matrix}$
	S.FW	$\underset{FW}{RUAV} = \begin{matrix}[12 & 12 & 12 & 2\pi]^T \\ \left[10 & \frac{4\pi}{9} & 2\pi\right]^T\end{matrix}$

Table 5.5: Cost matrices for the fixed-wing w/ acceleration and RUAV differential game.

	FW UAV	RUAV
Q	diag([255,255,255,275,220,220,255,280])	diag(155,155,185,135)
P	10*Q	10*Q
R	diag([0.1,0.1,0.1,0.1,0.1])	diag([0.1,0.1,0.1,0.1])

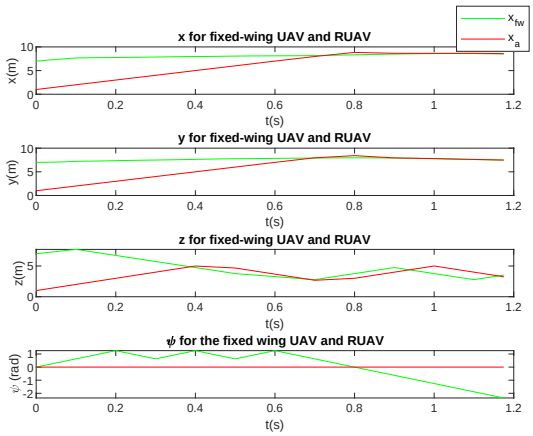


Figure 5.20: State evolution (FW UAV vs. RUAV) - Equal constraints

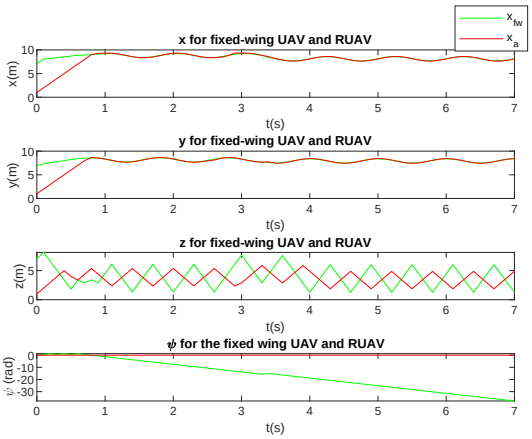


Figure 5.21: State evolution (FW UAV vs. RUAV) - Faster FW UAV

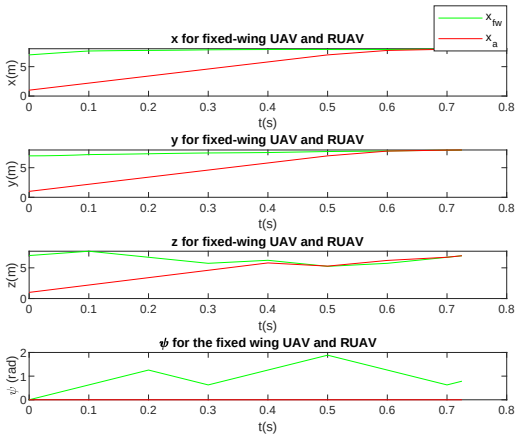


Figure 5.22: State evolution (FW UAV vs. RUAV) - Faster RUAV

chosen, the three position states and the yaw angle. This matrix is now given by

$$C_{fw} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

The input constraints are the same ones as in Table 5.4, where the change applied was to accommodate the change in the number of states of the fixed-wing UAV. Figure 5.38 and 5.39 shows the plot of the states for the scenario where the fixed-wing UAV has the advantage and the other where the RUAV has de advantage. We can see that in both scenarios the RUAV pursues the fixed-wing UAV very closely in the beginning, catching up and catching it in the second scenario. In case 1 we can also see that roughly at the halfway point the fixed-wing UAV manages to outmanoeuvre the other vehicle and put

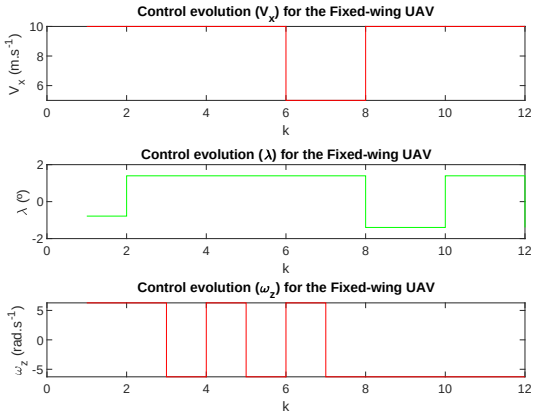


Figure 5.23: Input evolution (FW UAV) - Equal constraints

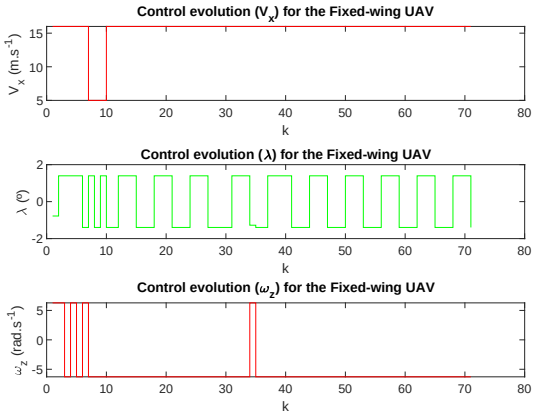


Figure 5.24: Input evolution (FW UAV) - Faster FW UAV

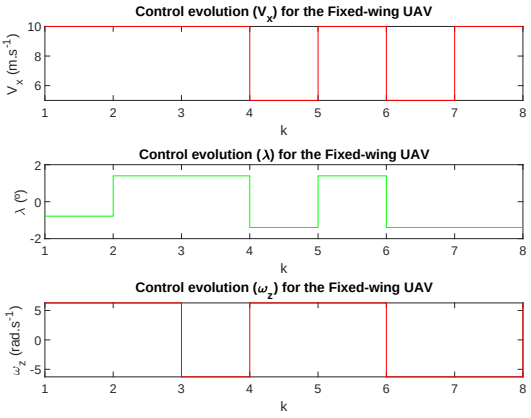


Figure 5.25: Input evolution (FW UAV) - Faster RUAV

Table 5.6: Input constraints (fw UAV w/ acceleration vs. RUAV).

		RUAV	FW UAV w/ acceleration
Lower bounds	F. FW	[-10 -10 -10 -2π]	[-2π -2π -8 -8 -8]
	S. FW	[-16 -16 -16 -2π]	[-2π -2π -8 -8 -8]
Upper bounds	F. FW	[10 10 10 2π]	[2π 2π 8 8 8]
	S. FW	[16 16 16 2π]	[2π 2π 8 8 8]

some distance between each other. Figures 5.40 and 5.41 show the input evolution of the FW while 5.42 and 5.43 show the input evolution of the RUAV. We can see that the inputs still saturate often, which means that the best solution for this particular problem might be to add another integral layer, where the inputs would now be variation of inputs, allowing us to control the jumps between each value. However, we can see

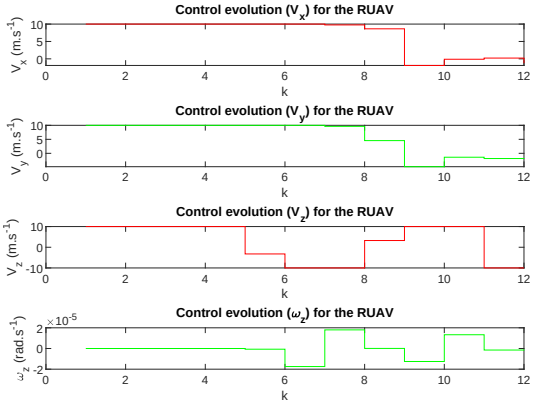


Figure 5.26: Input evolution (RUAV) - Equal constraints

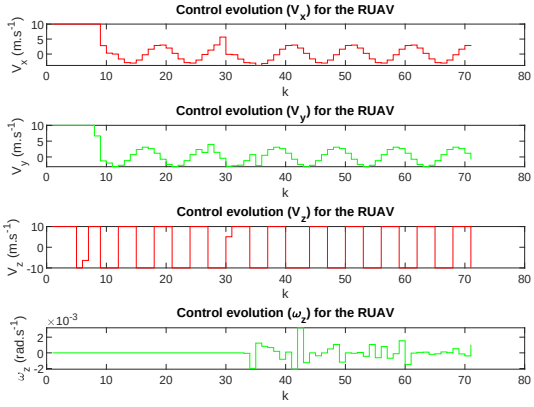


Figure 5.27: Input evolution (RUAV) - Faster FW UAV

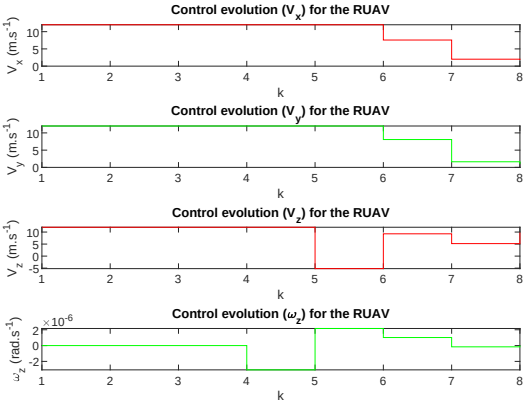


Figure 5.28: Input evolution (RUAV) - Faster RUAV

some smoothness in both, especially in the pursuer, which might indicate that adding complexity to the vehicles might also help in this issue. The easiest and most intuitive way to see the difference in smoothness is the position plot which can be seen in Figures 5.44 and 5.45. When compared with Figures 5.30 and 5.31, scenario 2 and 3, we can see that the fixed-wing UAV moves less erratically, making less jagged lines, while the behaviour of the RUAV stays roughly the same. When looking at the plot for each iterations time we can see that the RUAV is still taking more time than the fixed-wing UAV, on average. Since the other vehicle is more complex, there are more considerations to make, leading to a longer time for each iteration, as seen in Figures 5.46 and 5.47. In table 5.7 the sum of each vehicle's iterations and the total simulation time are shown. The last plot we can look at the distance between each vehicle along the simulation. This was calculated using Equation 5.7, and the results can be seen in Figures 5.48 and 5.49. The most interesting thing to see is that around three seconds the fixed-wing makes an escape and beyond that

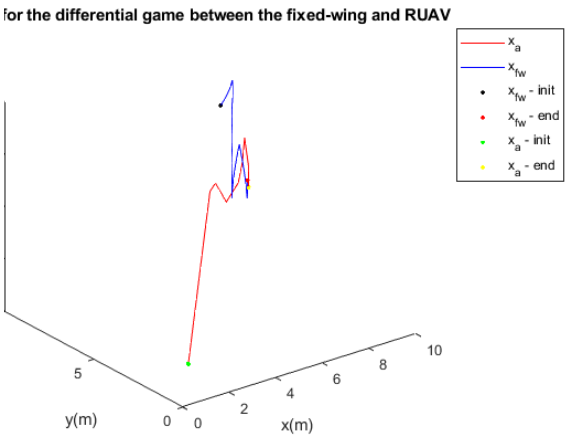


Figure 5.29: XYZ plot
(FW UAV vs. RUAV) - Equal constraints

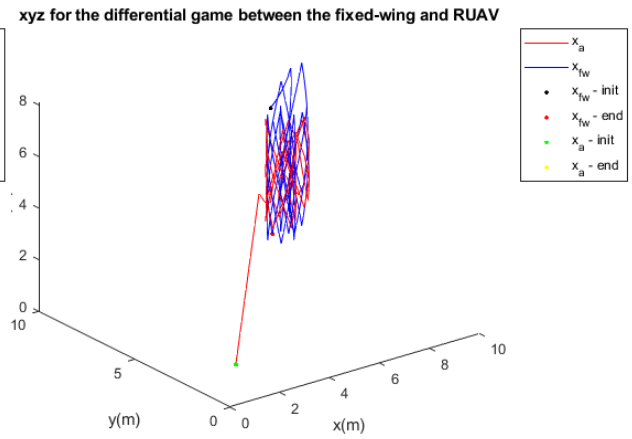


Figure 5.30: XYZ plot
(FW UAV vs. RUAV) - Faster FW UAV

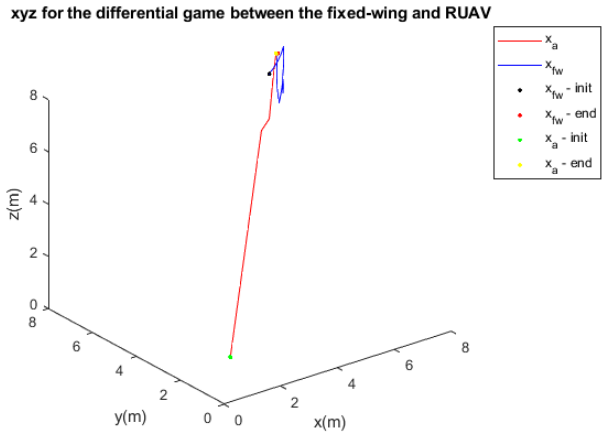


Figure 5.31: XYZ plot (FW UAV vs. RUAV) - Faster RUAV

the distance between the vehicles only grows. Lastly it is good to analyse the runtime of each simulation, Table 5.7 shows these values. Two things are very noticeable. The first is that the runtimes jump to a higher value when we pass from the ground vehicle to the fixed-wing UAV. The latter takes much more time to catch than the first. The second is that the sum of all the iterations of both vehicles diminishes with the fixed-wing, just like observed before.

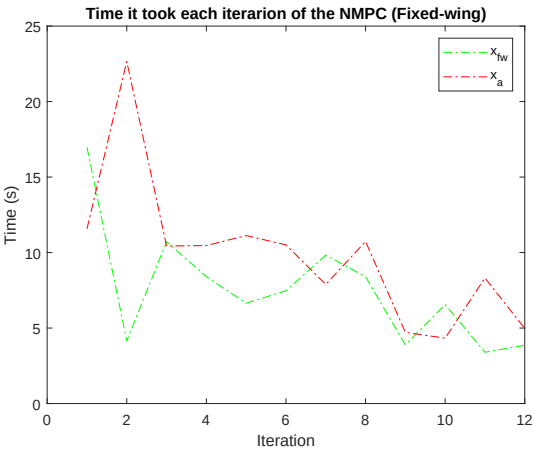


Figure 5.32: Time of each iteration (FW UAV vs. RUAV) - Equal constraints

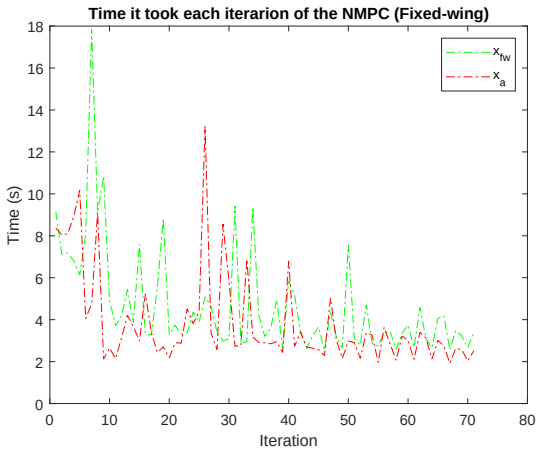


Figure 5.33: Time of each iteration (FW UAV vs. RUAV) - Faster FW UAV

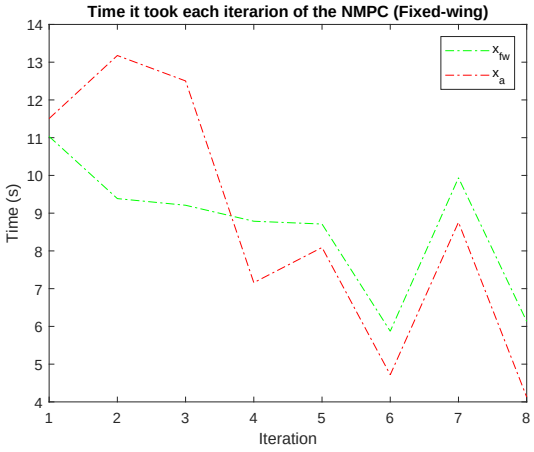


Figure 5.34: Time of each iteration (FW UAV vs. RUAV) - Faster RUAV

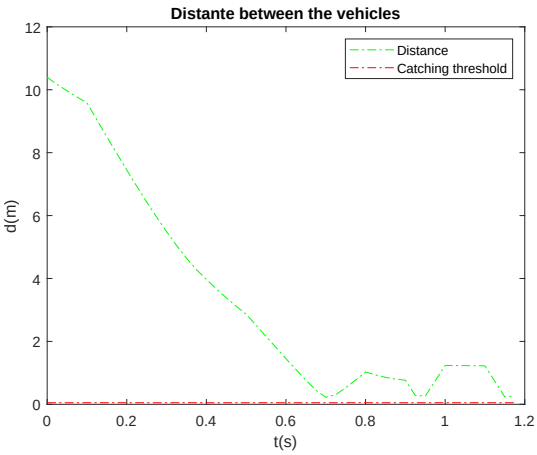


Figure 5.35: Distance between vehicles (FW UAV vs. RUAV) - Equal constraints

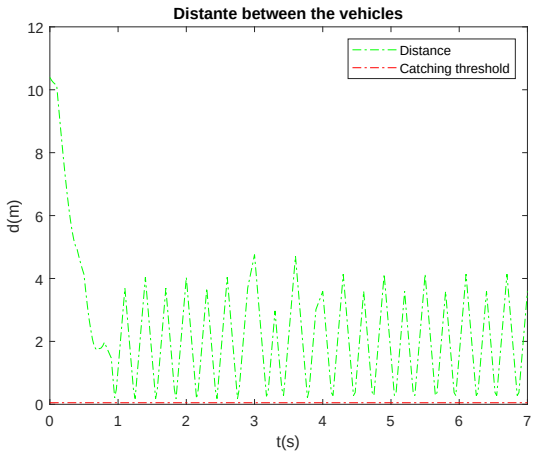


Figure 5.36: Distance between vehicles (FW UAV vs. RUAV) - Faster FW UAV

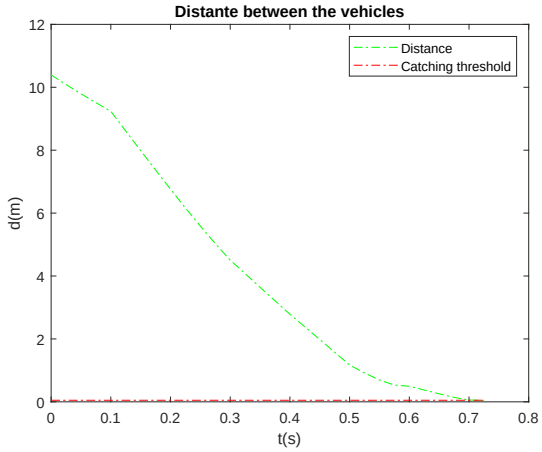


Figure 5.37: Distance between vehicles (FW UAV vs. RUAV) - Faster RUAV

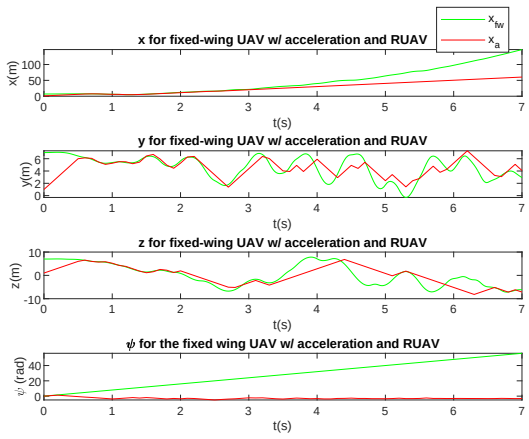


Figure 5.38: Position and yaw angle plot (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

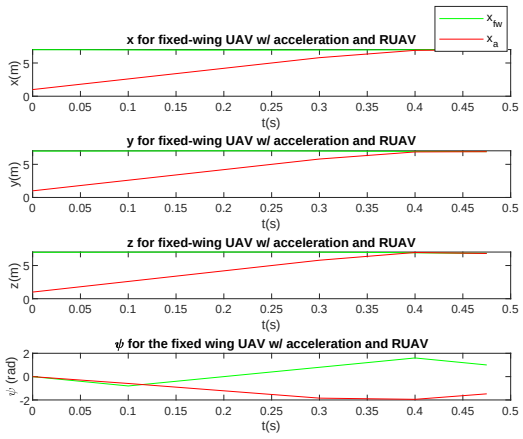


Figure 5.39: Position and yaw angle plot (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

Table 5.7: Total and individual runtimes for the differential game

	UGV	FW	FW w/ acceleration
Case 1 (E.v)	UGV -> ~54s	FW -> ~594s	N/A
	UAV -> ~67s	UAV -> ~666s	N/A
	Total-> ~125s	Total -> ~1265s	N/A
Case 2 (UGV/FW.f)	UGV -> ~371s	FW -> ~766s	FW -> ~1223s
	UAV-> ~403s	UAV -> ~822s	UAV -> ~1348s
	Total-> ~778s	Total -> ~1592s	Total -> ~2573s
Case 3 (UGV/FW.s)	UGV -> ~63s	FW -> ~106s	FW -> ~289s
	UAV-> ~76s	UAV -> ~108s	UAV -> ~316s
	Total-> ~141s	Total -> ~218s	Total -> ~606s

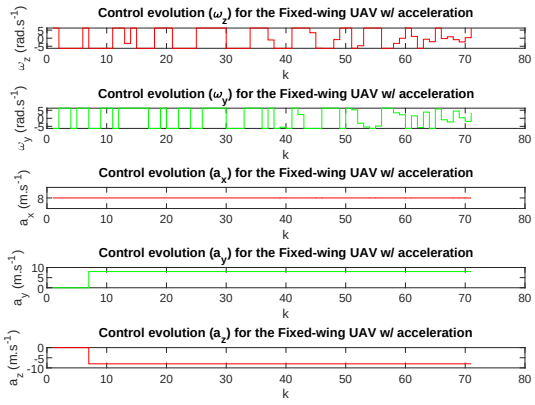


Figure 5.40: Input Evolution of the Fixed-Wing (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

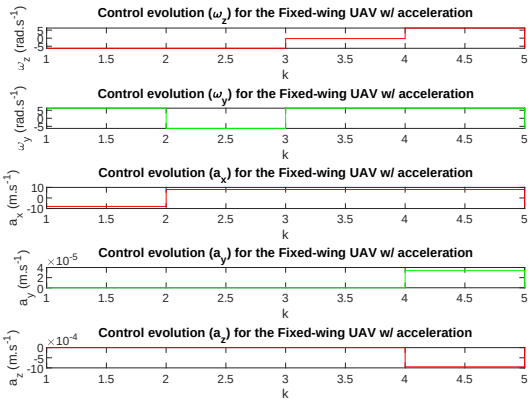


Figure 5.41: Input Evolution of the Fixed-Wing (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

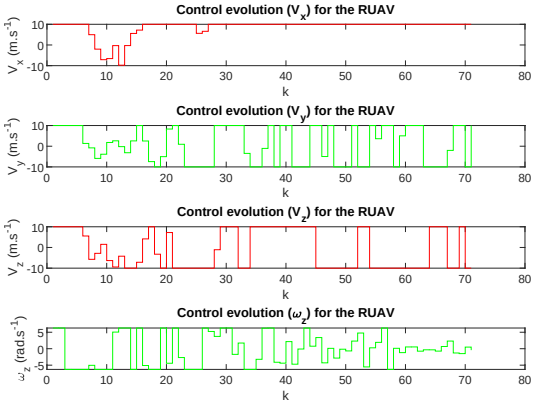


Figure 5.42: Input Evolution of the RUAV (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

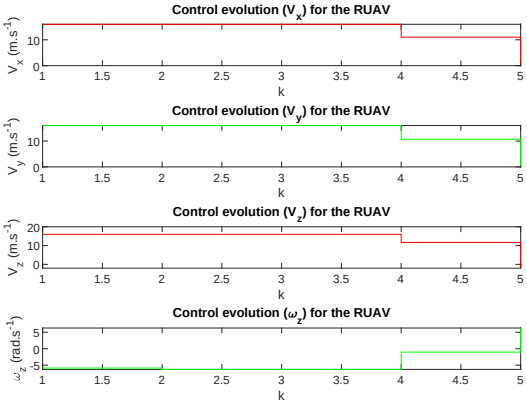


Figure 5.43: Input Evolution of the RUAV (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

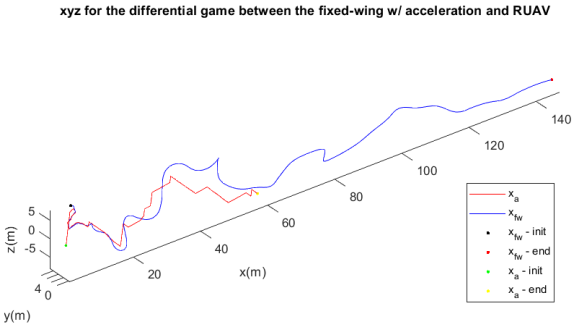


Figure 5.44: XYZ plot (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

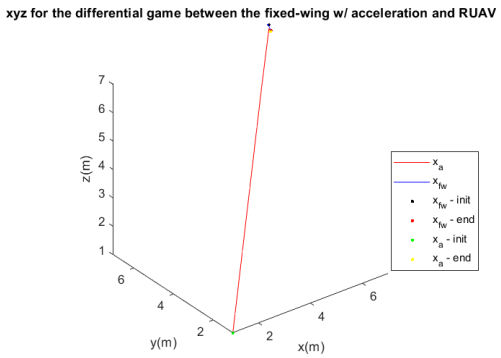


Figure 5.45: XYZ plot (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

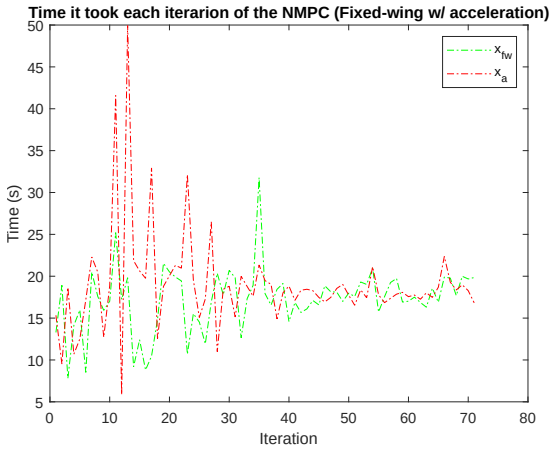


Figure 5.46: Time of each iteration (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

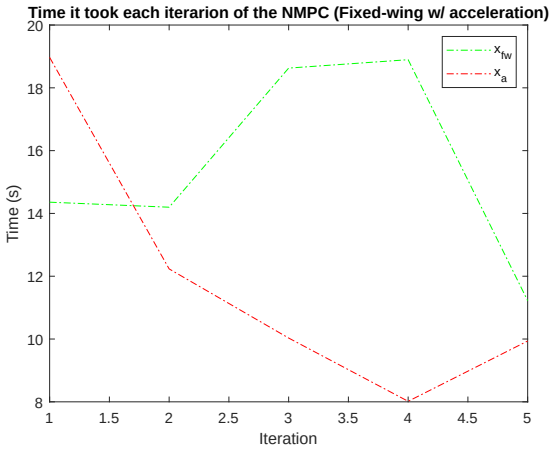


Figure 5.47: Time of each iteration (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

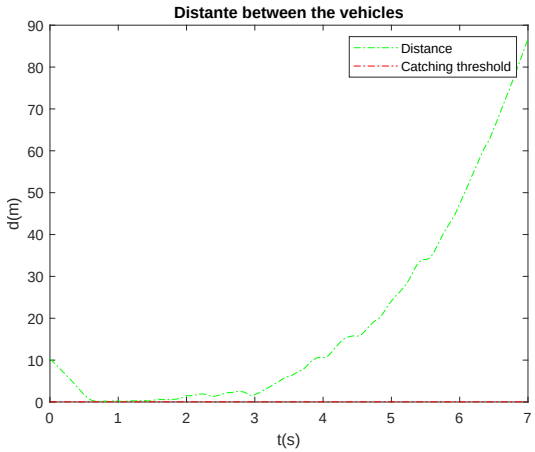


Figure 5.48: Distance between vehicles (FW UAV w/ acceleration vs. RUAV) - Faster Fixed-Wing

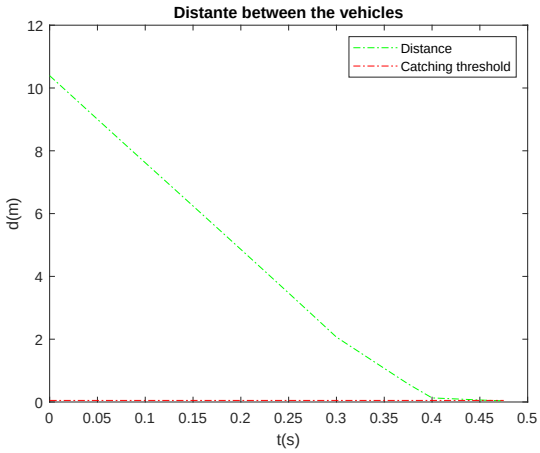


Figure 5.49: Distance between vehicles (FW UAV w/ acceleration vs. RUAV) - Faster RUAV

6

Human-in-the-loop

As was mentioned in Chapter 2 the two compensatory models presented were the extended crossover model which is defined by

$$Y_c(s) = K_c \frac{T_L s + 1}{T_I s + 1} e^{-s\tau_c} \tag{6.1}$$

where T_L and T_I are time constants of the equalization component. For the simplified precision model, which includes a neuro-muscular dynamic, we have

$$Y_{nm}(s) = \frac{\omega_{nm}^2}{s^2 + 2\alpha_{nm}\omega_{nm}s + \omega_{nm}^2} \tag{6.2}$$

in which ω_{nm} is the natural frequency and α_{nm} is the damping factor. These two models are then connected to form the full compensatory model whose Laplace transform is given by

$$Y_c(s) = K_c' \frac{T_L s + 1}{T_I s + 1} e^{-s\tau_c} Y_{nm}(s) \tag{6.3}$$

For this thesis the human modelling was done by introducing a reaction time constant as a first order model. This is a simplistic version of modelling the human so that the NMPC has a straightforward notion of the disadvantage of HIL, which can still be identified in the future with real data of human piloting. Considering all this, the model created has no intention of being a complete description of the human being. Its Laplace transform is given by

$$x_u = \frac{u}{T_H s + 1} \tag{6.4}$$

where the value of T_H represents the response time associated with the delay caused by the difference between the brain processing stimuli and the subsequent action and u is the input vector. Although this model is not enough to fully constitute a human being it is already representative of some actions one would take. When applied to our system

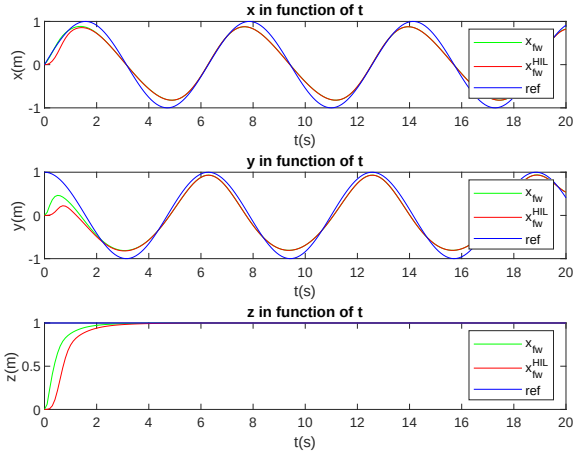


Figure 6.1: State evolution of the position for the NMPC with HIL

and $x_u = \begin{bmatrix} a^H_{fw} & \omega^H_{fw} \end{bmatrix}$ is added as a state, the model equation turns into

$$\begin{aligned} \dot{P}_{fw} &= R_{fw}(\lambda) V_{fw} \\ \dot{V}_{fw} &= a^H_{fw} \\ \dot{\lambda}_{fw} &= \omega^H_{fw} \\ \dot{x}_{u_{fw}} &= -\frac{1}{T_H} x_{u_{fw}} + \frac{1}{T_H} u_{fw} \end{aligned} \tag{6.5}$$

where the first three states are the same as in Equation (3.34) except that the input are now coming from x_u and not u, and the fourth state is the dynamic that represents a human being within the model. In terms of the optimal control problem itself no changes were performed, since the new states aren't factored in except for the changes applied to the model to tranform it into a HIL model.

6.1 NMPC for a fixed-wing UAV

Using the model described above and the same Optimal Control Problem (OCP) developed in Chapter 4, in the NMPC as well as the system, we can obtain results for the simulation of the NMPC seen in Section 4.1.2 with a human-in-the-loop. The gains were kept the same in order to compare the control in the two cases. For this the time T_H considered was 0.2 seconds in accordance with the literature mentioned in Chapter 2, where we consider a slightly faster human then the ones studied. Figure 6.1 shows the states for the position of the NMPC with and without HIL for the fixed-wing UAV with acceleration. We can see that when the HIL is applied the fixed-wing UAV is slower to arrive at the reference, having a similar value than the simulation without HIL, only a few instances later, as was expected. For the velocity states we can see that for the linear velocity in the x-axis, the simulation with HIL seems to be closer to the reference while in

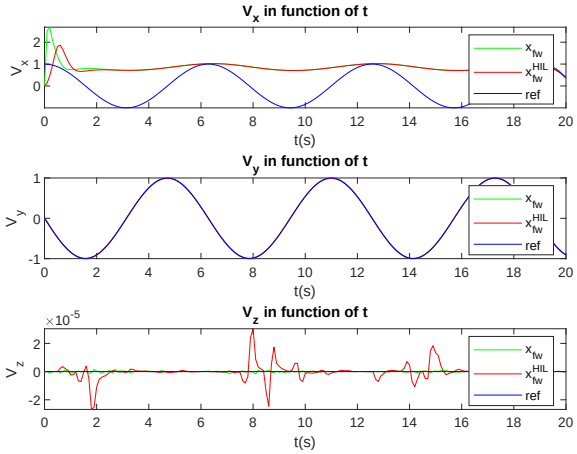


Figure 6.2: State evolution of the velocity for the NMPC with HIL

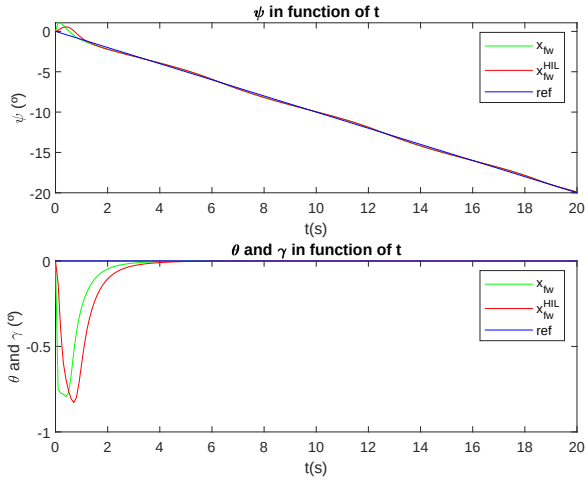


Figure 6.3: State evolution of the angles for the NMPC with HIL

the z-axis it becomes more erratic, as shown in Figure 6.2, this can be caused by an undetected inconsistency with the OCP, which is propagated throughout the results. The same observation can be applied to Figure 6.3, where for both the pitch and flight-path, for the RUAV and fixed-wing UAV respectively, and yaw angles, the simulation that considers the human has a slower performance. Figures 6.4 and 6.5 show the input evolutions for both scenarios. Here we can see that the HIL introduces a degradation of performance. To show this more intuitively the 3D plot was made as can be seen in Figure 6.6, where we can see that there are points where the angles differ from the reference. Finally, we will look at the time it took for each iterations to run, as seen in Figure 6.7. The sum of all iterations is roughly 1730 seconds. When we compare this to the other NMPC simulations we can see that the time it takes has increased considerably.

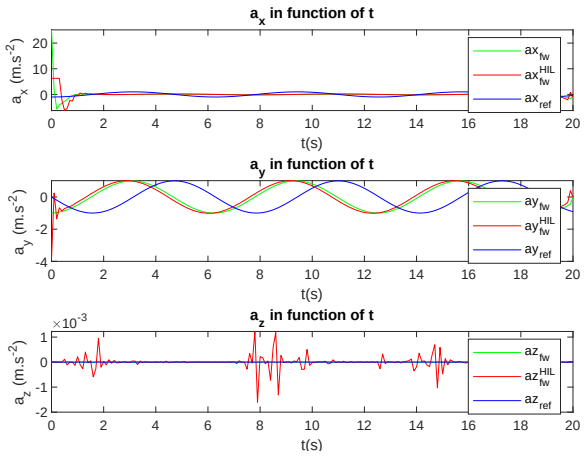


Figure 6.4: Input evolution of the acceleration for the NMPC with HIL

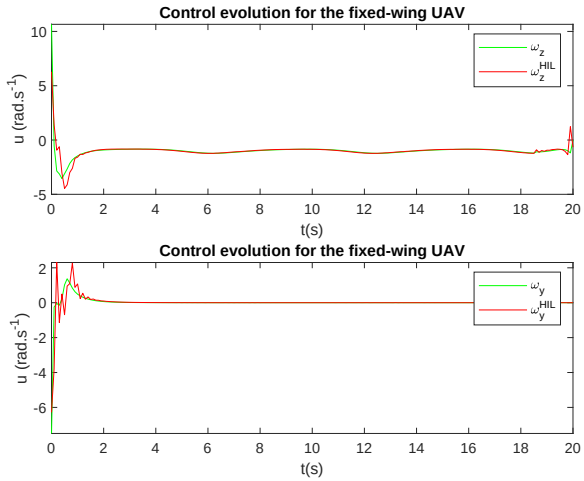


Figure 6.5: Input evolution of the angular velocities for the NMPC with HIL

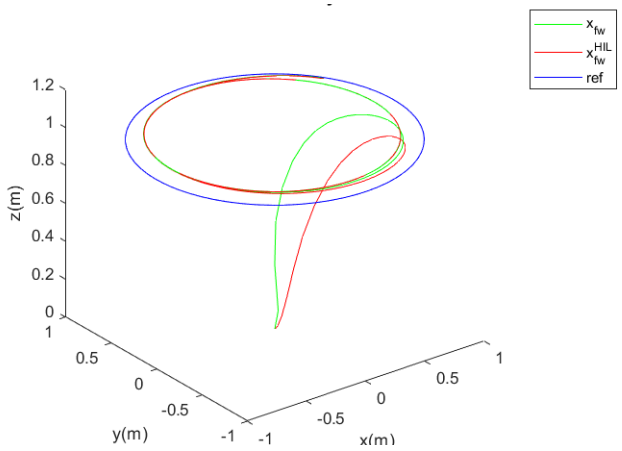


Figure 6.6: 3D plot for the NMPC with HIL

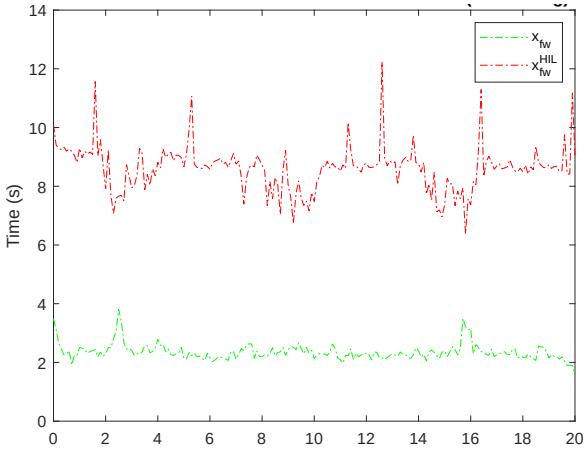


Figure 6.7: Time of each iteration for the NMPC with HIL

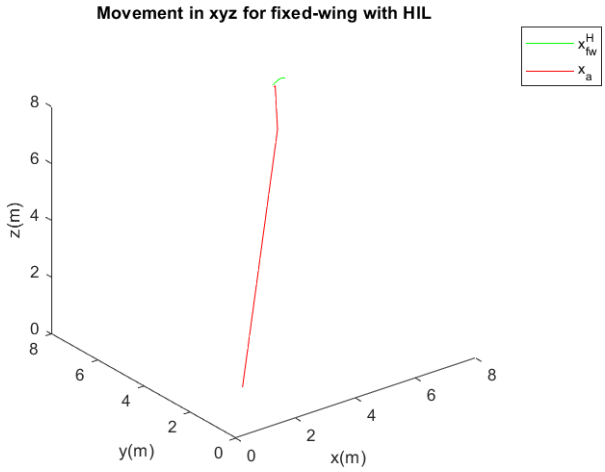


Figure 6.8: XYZ plane for the differential game with HIL (Unchanged values)

6.2 Pursuit-evasion Game with HIL

Once again, the same process was applied for the differential game, taking into account that both controllers have to calculate the optimization for the fixed-wing. Considering the same matrix values of Table 5.5 where the fixed-wing won, we can see that the human controlled fixed-wing gets caught almost immediately, as seen in Figure 6.8. This plot shows that adding the human is disadvantageous at first sight, causing the RUAV to pursue the other vehicle more efficiently. So that the behaviour of the vehicles could be seen in the case where the human controlled vehicle could escape the input constraints were changed. In this simulation the fixed-wing vehicle starts in the same position and the same cost matrix gains but the fixed-wing drone has two times less constraints which are now $\pm\begin{bmatrix}4\pi & 4\pi & 15 & 15 & 15\end{bmatrix}^T$. In Figure 6.9 we can see that RUAV follows the

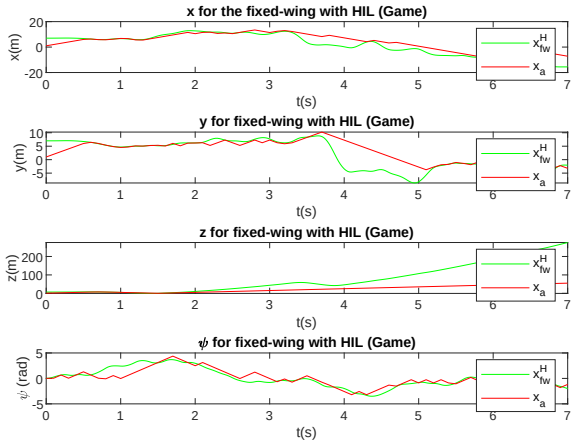


Figure 6.9: State evolution for the differential game with HIL

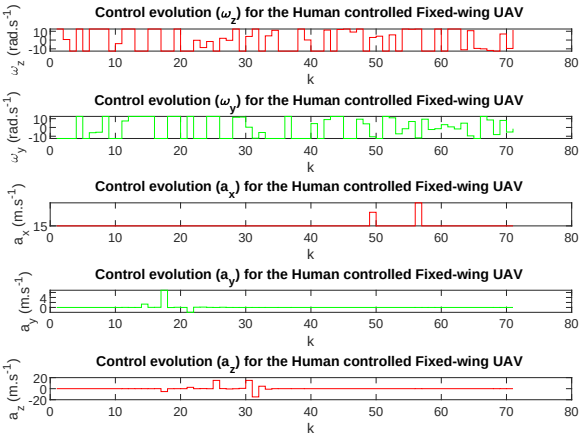


Figure 6.10: Input evolution of the fixed-wing vehicle for the differential game with HIL

fixed-wing very closely but that around the halfway point it manages to escape in the z-axis. Figures 6.10 and 6.11 show the input evolutions of both vehicles. We can see that for the fixed-wing inputs, they stay saturated throughout the simulation, continuously going at full throttle. For the RUAV we start to notice some less saturations, however the results are still very erratic. In Figure 6.12 we can see the movement of both vehicles in their chase. We can see that at some point the fixed-wing escapes up in the z-axis not being accompanied by the RUAV. Once again it is important to notice the difference in behaviour between the vehicles. While the RUAV lines are more jagged, the fixed-wing movement as developed with more curves. Figure 6.13 shows the distance between the vehicles, calculated using Equation (5.7). As it was expected the vehicles stay close to one another until roughly the halfway point, and then the distance gets bigger until the end of the simulation. Finally, we will see the plot for the time of each iteration (Figure 6.14). We can see that both vehicles' times as increasing considerably in comparison but that both stay close to one another, and we do not have one vehicle taking much more time than the other. It is important to note that with different optimization libraries, such as,

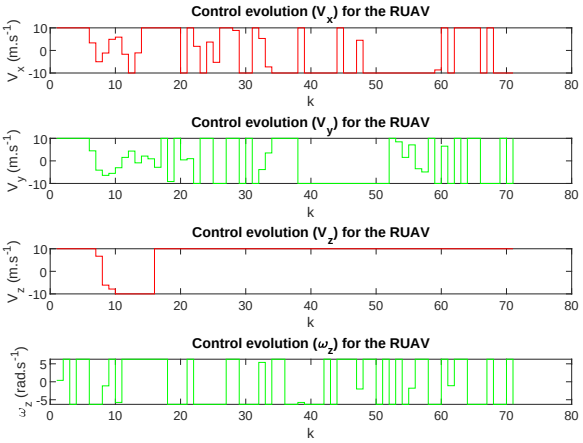


Figure 6.11: Input evolution of the RUAV for the differential game with HIL

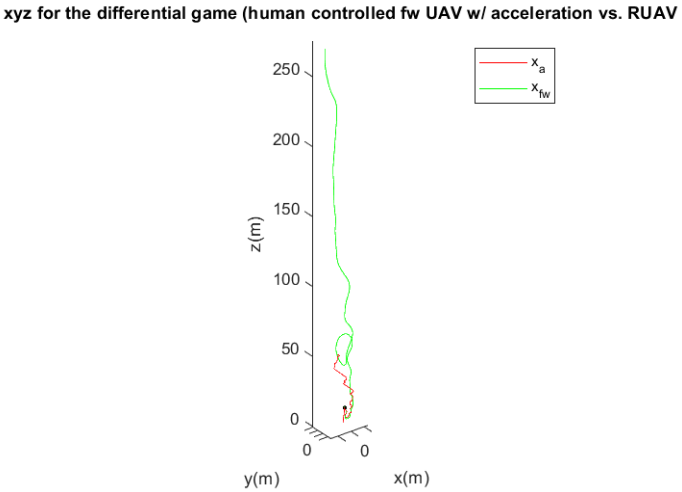


Figure 6.12: XYZ plot for the differential game with HIL

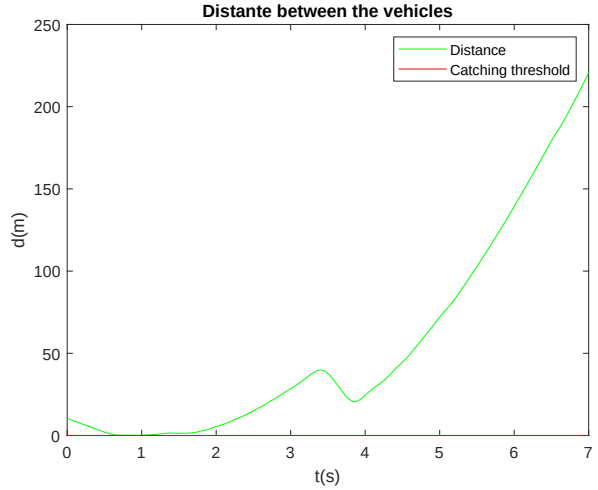


Figure 6.13: Distance between vehicles for the differential game with HIL

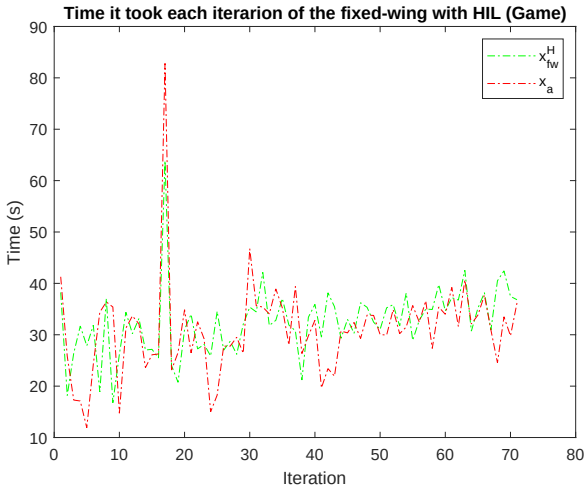


Figure 6.14: Time of each iteration for the differential game with HIL

YALMIP, CASADI, or ACADOS, this problem will be able to be implemented in real-time.

7

Conclusions and Future work

This research aimed to create and simulate an unmanned aerial vehicle capable of following and capturing a fixed-wing manned drone by designing a non-linear predictive controller able to react to unpredictable and fast changing trajectories, and a differential game in order to create a competitive environment of pursuit-evasion.

The thesis starts by analysing and reviewing various studies on the different subjects of this dissertation. In drone control many types of controllers can be implemented, such as PID controllers. Ultimately the NMPC was picked considering that for the objective of this thesis having a predictive element to the control would be important. After that it was necessary to pick the manner with which we would be representing the human in the control loop, which could be done in various ways. In the end a first order system was chosen which, although not representing the human being to its full capacity and ingenuity, takes into considerations the delay caused by the reaction time of the pilot.

Following this step, the target and shuttle drone models are developed, through various levels of complexity, as well as some internal controllers for the RUAV. For the target models we could see that the UGV was able to obtain good results but was unable to follow a reference without a controller. The first internal controller simulated for the RUAV shows a good tracking of the Euler angles, but no way of following a given position reference. The Mellinger controller shows much more promise, being able to follow a reference closely, being it the simplified or the complete version. In this chapter the development of simplified models for the aerial vehicles was also done since both vehicles are very non-linear and complex which would add much computational weight to the simulations. Since the priority is to study the movement of the vehicles in the pursuit-evader game, simplified models were considered instead.

A NMPC controller is developed and tested in order to ascertain its robustness and tracking abilities. The results show that both the UGV and RUAV NMPC controllers are efficient at following the reference, while the fixed-wing demonstrates a worse performance. When developing the non-linear predictive controller the MATLAB function used was `fmincon` which is a simple and good tool to use but not optimized, causing the simulation to run for even more time that could be reduced.

After the controllers were simulated and had a good performance the next step was to create the pursuit-evasion game between the target vehicle and shuttle drone. The results show that when both vehicles have balanced capabilities or the shuttle drone has advantage, the latter vehicle catches its target easily, whereas when the target vehicle has the advantage it is able to escape. An issue that was noticed is that considering the cost used to create this game the vehicles slowed down their response if they got close, in the case of the RUAV, or if they were very far apart, in the case of the target vehicle. A way to circumvent this is adding a fixed parameter to the cost so that both vehicles always tried to do their role to the best of their efforts. A final observation in this chapter is the corroboration that the complexity of the vehicles impacts the runtime of each simulation since the more complex the model got, the more time the simulation would take to complete.

Finally, the human factor was added to the control loop using a simplified model of the typical reaction time. We can see that if both vehicles are balanced the target vehicle being operated by the human does not stand a chance with the RUAV. For the simulation without the HIL, the RUAV has to decrease its constraints by 60% in order to catch the fixed-wing. With the HIL it does not need to change any values and is able to catch the fixed-wing easily. In order for the fixed-wing to win it is necessary to diminish the input constraints roughly 100%, enabling the fixed-wing to double its efforts to escape. This shows that including the delay has caused a worse performance to the target vehicle. However, it is important to note that using a more composite model to represent the human being might turn the disadvantage into an advantage.

7.1 Future Work

Based on these conclusions there are some areas that should be further investigated. Firstly, additional work should be developed so that both the NMPC controller and the differential game are able to control and use the complete RUAV and fixed-wing model, so that the full physics of the vehicles are represented. In the same vein, the modelling of the human being should be further studied so that both the behaviour and the resourcefulness of people are fully represented. There should be an effort in optimizing the MATLAB use such as changing from `fmincon` to libraries like `YALMIP`, `CASADI`, or `ACADOS`, which might use the same algorithms as `fmincon` but are applied in a much more efficient way. An interesting evolution of this work would be adding more pursuers, creating a team able to work together in order to catch the evader vehicle, using the control studied as well as a differential pursuit-evasion game that are able to consider more than two players to capture the target vehicle.

Bibliography

[1] S. HEANEY. "Drone sighting causes flights to be suspended at Dublin Airport". In: *Irish Examiner* (Jan. 25, 2023). URL: https://www.irishexaminer.com/news/arid-41056655.html (visited on 01/31/2023) (cit. on p. 1).

[2] A. N. da Aviação Civil. "Legislação e Regulamentação Específica". In: (Jan. 31, 2023). URL: https://www.anac.pt/vPT/Generico/drones/legislacao_regulamentacao_uas/Paginas/LegislacaoeRegulamentacaoEspecifica.aspx (visited on 01/31/2023) (cit. on p. 1).

[3] A. Restas et al. "Drone applications for supporting disaster management". In: *World Journal of Engineering and Technology* 3.03 (2015), p. 316 (cit. on p. 4).

[4] J. Shahmoradi et al. "A Comprehensive Review of Applications of Drone Technology in the Mining Industry". In: *Drones* 4.3 (2020). ISSN: 2504-446X. DOI: 10.3390/drones4030034. URL: https://www.mdpi.com/2504-446X/4/3/34 (cit. on p. 4).

[5] G. Szafranski and R. Czyba. "Different approaches of PID control UAV type quadrotor". In: (2011) (cit. on p. 4).

[6] H. Bouadi, M. Bouchoucha, and M. Tadjine. "Sliding mode control based on backstepping approach for an UAV type-quadrotor". In: *World Academy of Science, Engineering and Technology* 26.5 (2007), pp. 22–27 (cit. on p. 5).

[7] J. Eklund, J. Sprinkle, and S. Sastry. "Implementing and testing a nonlinear model predictive tracking controller for aerial pursuit/evasion games on a fixed wing aircraft". In: *Proceedings of the 2005, American Control Conference, 2005.* 2005, 1509–1514 vol. 3. DOI: 10.1109/ACC.2005.1470179 (cit. on p. 5).

[8] D. Q. Mayne. "Model predictive control: Recent developments and future promise". In: *Automatica* 50.12 (2014), pp. 2967–2986. ISSN: 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2014.10.128. URL: https://www.sciencedirect.com/science/article/pii/S0005109814005160 (cit. on p. 5).

[9] D. Limon et al. "Input to state stability of min–max MPC controllers for nonlinear systems with bounded uncertainties". In: *Automatica* 42.5 (2006), pp. 797–803. ISSN: 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2006.01.001. URL: https://www.sciencedirect.com/science/article/pii/S0005109806000483 (cit. on p. 5).

[10] ""Receding Horizon Control"". In: *"Optimal Control of Constrained Piecewise Affine Systems"*. "Berlin, Heidelberg": "Springer Berlin Heidelberg", "2007", "26–38". ISBN: "978-3-540-72701-9". DOI: "10.1007/978-3-540-72701-9_3". URL: https://doi.org/10.1007/978-3-540-72701-9_3 (cit. on p. 5).

[11] F. Künhe, J. Gomes, and W. Fetter. "Mobile robot trajectory tracking using model predictive control". In: *II IEEE latin-american robotics symposium*. Vol. 51. Citeseer. 2005 (cit. on p. 5).

[12] A. Afram and F. Janabi-Sharifi. "Theory and applications of HVAC control systems – A review of model predictive control (MPC)". In: *Building and Environment* 72 (2014), pp. 343–355. ISSN: 0360-1323. DOI: https://doi.org/10.1016/j.buildenv.2013.11.016. URL: https://www.sciencedirect.com/science/article/pii/S0360132313003363 (cit. on p. 5).

[13] Y. Yao and D. K. Shekhar. "State of the art review on model predictive control (MPC) in Heating Ventilation and Air-conditioning (HVAC) field". In: *Building and Environment* 200 (2021), p. 107952. ISSN: 0360-1323. DOI: https://doi.org/10.1016/j.buildenv.2021.107952. URL: https://www.sciencedirect.com/science/article/pii/S0360132321003565 (cit. on p. 5).

[14] G. Serale et al. "Model Predictive Control (MPC) for Enhancing Building and HVAC System Energy Efficiency: Problem Formulation, Applications and Opportunities". In: *Energies* 11.3 (2018). ISSN: 1996-1073. DOI: 10.3390/en11030631. URL: https://www.mdpi.com/1996-1073/11/3/631 (cit. on p. 5).

[15] S. Gros et al. "From linear to nonlinear MPC: bridging the gap via the real-time iteration". In: *International Journal of Control* 93.1 (2020), pp. 62–80. DOI: 10.1080/00207179.2016.1222553. eprint: https://doi.org/10.1080/00207179.2016.1222553. URL: https://doi.org/10.1080/00207179.2016.1222553 (cit. on p. 5).

[16] L. O. Santos et al. "On-line implementation of nonlinear MPC: an experimental case study". In: *Control Engineering Practice* 9.8 (2001). Advanced Control of Chemical Processes, pp. 847–857. ISSN: 0967-0661. DOI: https://doi.org/10.1016/S0967-0661(01)00049-1. URL: https://www.sciencedirect.com/science/article/pii/S0967066101000491 (cit. on p. 5).

[17] Z. Chao et al. "Collision-free UAV formation flight control based on nonlinear MPC". In: (2011), pp. 1951–1956. DOI: 10.1109/ICECC.2011.6066578 (cit. on p. 5).

[18] R. Findeisen et al. "State and Output Feedback Nonlinear Model Predictive Control: An Overview". In: *European Journal of Control* 9.2 (2003), pp. 190–206. ISSN: 0947-3580. DOI: https://doi.org/10.3166/ejc.9.190-206. URL: https://www.sciencedirect.com/science/article/pii/S0947358003702751 (cit. on pp. 6, 8).

[19] R. Findeisen and F. Allgöwer. "An introduction to nonlinear model predictive control". In: *21st Benelux meeting on systems and control*. Vol. 11. Citeseer. 2002, pp. 119–141 (cit. on pp. 7, 8).

[20] S. Gros and M. Zanon. "Data-Driven Economic NMPC Using Reinforcement Learning". In: *IEEE Transactions on Automatic Control* 65.2 (2020), pp. 636–648. DOI: 10.1109/TAC.2019.2913768 (cit. on p. 8).

[21] H. Han and J. Qiao. "Nonlinear Model-Predictive Control for Industrial Processes: An Application to Wastewater Treatment Process". In: *IEEE Transactions on Industrial Electronics* 61.4 (2014), pp. 1970–1982. DOI: 10.1109/TIE.2013.2266086 (cit. on p. 8).

[22] S. Munir et al. "Cyber Physical System Challenges for Human-in-the-Loop Control". In: *8th International Workshop on Feedback Computing (Feedback Computing 13)*. San Jose, CA: USENIX Association, June 2013. URL: https://www.usenix.org/conference/feedbackcomputing13/workshop-program/presentation/munir (cit. on p. 9).

[23] S. Munir et al. "Cyber Physical System Challenges for {Human-in-the-Loop} Control". In: *8th International Workshop on Feedback Computing (Feedback Computing 13)*. 2013 (cit. on p. 9).

[24] R. Chipalkatty et al. "Human-in-the-loop: MPC for shared control of a quadruped rescue robot". In: *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2011, pp. 4556–4561 (cit. on p. 9).

[25] W. Takano et al. "Recognition of human driving behaviors based on stochastic symbolization of time series signal". In: *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2008, pp. 167–172 (cit. on p. 9).

[26] C.-P. Lam and S. S. Sastry. "A POMDP framework for human-in-the-loop system". In: *53rd ieee conference on decision and control*. IEEE. 2014, pp. 6031–6036 (cit. on p. 9).

[27] F. Broz, I. Nourbakhsh, and R. Simmons. "Planning for human–robot interaction in socially situated tasks". In: *International Journal of Social Robotics* 5.2 (2013), pp. 193–214 (cit. on p. 10).

[28] D. S. Nunes, P. Zhang, and J. S. Silva. "A survey on human-in-the-loop applications towards an internet of all". In: *IEEE Communications Surveys & Tutorials* 17.2 (2015), pp. 944–965 (cit. on p. 10).

[29] C. Cimini et al. "A human-in-the-loop manufacturing control architecture for the next generation of production systems". In: *Journal of manufacturing systems* 54 (2020), pp. 258–271 (cit. on p. 10).

[30] W. Li et al. "Synthesis for human-in-the-loop control systems". In: *International conference on tools and algorithms for the construction and analysis of systems*. Springer. 2014, pp. 470–484 (cit. on p. 10).

[31] W. Yin et al. "Effects of trust in human-automation shared control: A human-in-the-loop driving simulation study". In: *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*. IEEE. 2021, pp. 1147–1154 (cit. on p. 10).

[32] F. M. Drop. *Control-Theoretic Models of Feedforward in Manual Control*. Vol. 46. Logos Verlag Berlin GmbH, 2016 (cit. on p. 10).

[33] "A comparative study of visual and auditory reaction times on the basis of gender and physical activity levels of medical first year students". In: *International journal of applied and basic medical research* 5.2 (2015), pp. 124–127. DOI: https://doi.org/10.4103/2229-516X.157168 (cit. on p. 10).

[34] A. Friedman. *Differential games*. Courier Corporation, 2013 (cit. on p. 11).

[35] G. Tzannetos, P. Marantos, and K. J. Kyriakopoulos. "A competitive differential game between an unmanned aerial and a ground vehicle using model predictive control". In: *2016 24th Mediterranean Conference on Control and Automation (MED)*. 2016, pp. 1053–1058. DOI: 10.1109/MED.2016.7535979 (cit. on pp. 11, 15, 20).

[36] A. Perelman, T. Shima, and I. Rusnak. "Cooperative differential games strategies for active aircraft protection from a homing missile". In: *Journal of Guidance, Control, and Dynamics* 34.3 (2011), pp. 761–773 (cit. on p. 11).

[37] L. A. Petrosjan. "Cooperative differential games". In: *Advances in dynamic games*. Springer, 2005, pp. 183–200 (cit. on p. 11).

[38] S. Clemhout and H. Y. Wan Jr. "Differential games—Economic applications". In: *Handbook of Game Theory with Economic Applications* 2 (1994), pp. 801–825 (cit. on p. 11).

[39] R. Vidal et al. "Probabilistic pursuit-evasion games: theory, implementation, and experimental evaluation". In: *IEEE Transactions on Robotics and Automation* 18.5 (2002), pp. 662–669. DOI: 10.1109/TRA.2002.804040 (cit. on p. 11).

[40] W. Lin. "Distributed UAV formation control using differential game approach". In: *Aerospace Science and Technology* 35 (2014), pp. 54–62. ISSN: 1270-9638. DOI: https://doi.org/10.1016/j.ast.2014.02.004. URL: https://www.sciencedirect.com/science/article/pii/S1270963814000376 (cit. on p. 11).

[41] C. R. Inc. *Jackal (Unmanned ground vehicle)*. URL: https://clearpathrobotics.com/jackal-small-unmanned-ground-vehicle/ (cit. on p. 13).

[42] F. Matos and B. Guerreiro. "Model predictive control strategies for parcel relay manoeuvres using drones". In: *2021 International Young Engineers Forum (YEFECE)*. IEEE. 2021, pp. 32–37 (cit. on p. 16).

[43] D. Mellinger and V. Kumar. "Minimum snap trajectory generation and control for quadrotors". In: *2011 IEEE International Conference on Robotics and Automation*. 2011, pp. 2520–2525. DOI: 10.1109/ICRA.2011.5980409 (cit. on p. 16).

[44] G. Dudek and M. Jenkin. *Computational principles of mobile robotics*. Cambridge university press, 2010 (cit. on p. 19).

[45] R. Carona, A. P. Aguiar, and J. Gaspar. "CONTROL OF UNICYCLE TYPE ROBOTS Tracking, Path Following and Point Stabilization". In: Nov. 2008 (cit. on p. 19).

[46] *Mobile Robot Kinematics*. URL: https://stanfordasl.github.io/aa274a/pdfs/notes/lecture1.pdf (cit. on p. 19).

[47] T. Magnusson. *Attitude control of a hexarotor*. 2014 (cit. on p. 21).